

AWS AI Practioner [AIF-01]

[통계](#) [수정](#) [삭제](#)

HyunJin Na · 8분 전

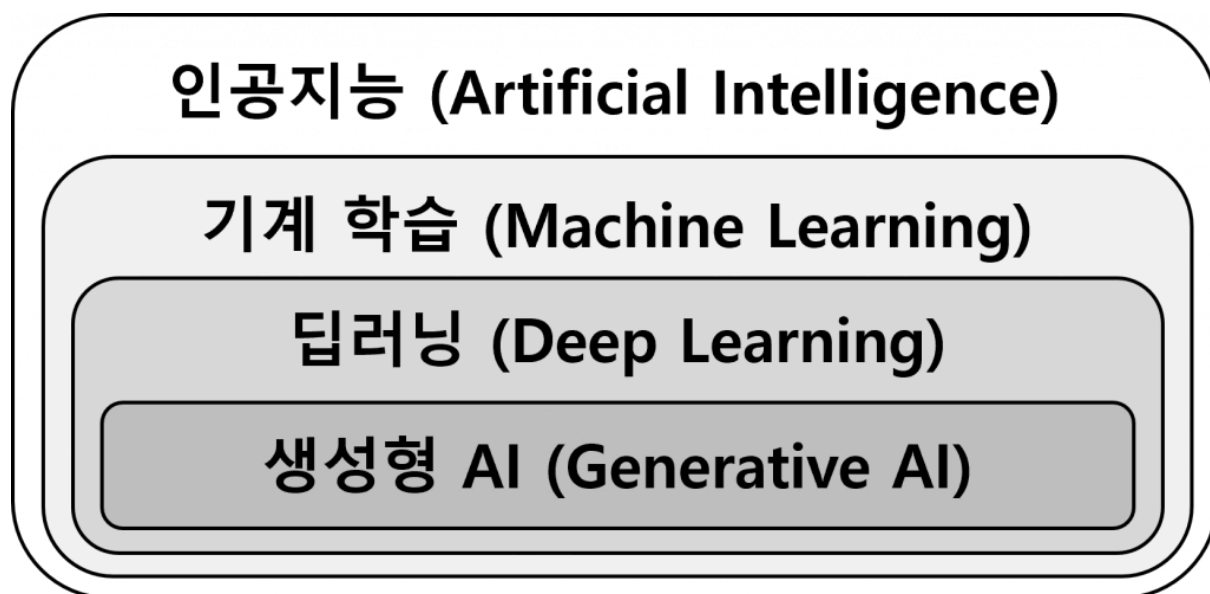
❤️ 0

AWS AI Practioner [AIF-01]

[제1장: AI, ML, DL, 그리고 Generative AI]

이 네 가지는 별개의 기술이 아니라, 큰 인형 안에 작은 인형이 들어있는 '러시아 마트료시카 인형' 같은 관계입니다.

인공 지능 > 머신 러닝 > 딥러닝 > 생성형 AI



1. 인공지능 (AI: Artificial Intelligence)

- **정의:** 인간의 지능(학습, 추론, 문제 해결)을 흉내 내는 **가장 넓은 범위**의 기술.
- **특징:** 꼭 데이터를 스스로 학습하지 않아도 됩니다. 사람이 정해진 규칙(If-Then)대로 움직이는 체스 프로그램도 AI에 해당합니다.
- **시험 키워드:** "인간 지능 모방", "가장 포괄적인 개념"

2. 머신러닝 (ML: Machine Learning)

- **정의:** 데이터를 통해 기계가 스스로 패턴을 학습하게 하는 AI의 하위 분야.
- **특징:** 사람이 일일이 규칙을 코딩하지 않습니다. 대신 수많은 데이터를 던져주면 기계가 "아, 이런 특징이 있으면 고양이구나!"라고 스스로 규칙을 찾아냅니다. 하지만 어떤 특징이 중요한지는 사람이 알려줘야 함 >> 이것이 딥러닝과의 차이
 - c.f) 딥러닝: 특징 추출까지 다층 레이어들이 스스로해서 로 데이터를 던져줘도 알아서 함
- **수명주기:** 비즈니스 목표 식별 > 머신러닝 문제 정의 > 데이터 처리(수집, 전처리, 추출) > 모델 개발(학습, 튜닝 및 평가) > 모델 배포(추론 및 예측) > 모델 모니터링 > 모델 재학습
- **정답 특징:**

문제 유형	정답의 특징 (Label)	예시 (Example)
이진 분류 (Binary)	정답이 딱 2개 중 하나 (Yes/No)	스팸인가? (Yes / No)
다중 클래스 (Multi-class)	정답 후보가 여러 개지만 선택은 단 하나 (클래스 =정답이라는 뜻)	사진 속 동물이 개, 고양이, 사자 중 무엇인가?
다중 레이블 (Multi-label)	한 데이터에 정답이 여러 개일 수 있음	이 영화의 장르는 액션이면서 코미디이다.
회귀 (Regression)	정답이 분류가 아니라 연속된 숫자	내일 주가가 75,500원일 것이다.

- **주요 프로세스 (시험 단골!):** 수집된 훈련 데이터는 머신러닝 알고리즘에 입력되며, 머신러닝 학습 과정은 전통적으로 3가지 범주로 나뉨

1. 지도 학습 (Supervised)

- 상태: 문제와 정답(Label)을 같이 줍니다. (예: 사진 + "이건 개임")
- 핵심: 바로 **Fine-tuning(미세 조정)**이 지도학습에 해당함!
- 종류:
 - **회귀(Regression):** 숫자 맞추기, 주가, 온도처럼 연속된 숫자를 맞추는 것. (예. 가격 예측)
 - **분류(Classification):** 스팸이냐 아니냐, 개냐 고양이냐처럼 카테고리를 나누는 것

2. 자기 지도 학습(Self-Supervised): FM 모델 "데이터 안에 정답이 숨어있다", 스스로 라벨을 붙임

- 상태: 라벨 없는 대량의 텍스트 (예: 인터넷의 모든 글)
- 학습 방식: 문장에서 단어 하나를 가리고 "여기 들어갈 말이 뭐게?"라고 스스로 문제를 내고 스스로 맞힘. 데이터 스스로가 가짜 정답을 만들어 스스로 학습함
- 핵심: 사람이 라벨을 안 달아줘도 데이터 자체가 정답(Implicit Label)이 됨. 사람이 정답지를 주지는 않지만, 데이터 안에서 스스로 정답을 창조해서 지도학습처럼 행동함 >> FM(파운데이션 모델)이 천재가 되는 비결!
 - c.f) Fine-Tuning이 labeled data로 지도학습을 하는 것

구분	Pre-training (FM 생성)	Fine-tuning (모델 최적화)
데이터 종류	Unlabeled (대량)	Labeled (소량)

구분	Pre-training (FM 생성)	Fine-tuning (모델 최적화)
학습 방식	자기 지도 학습 (Self-supervised)	지도 학습 (Supervised)
가중치 변화	무에서 유를 창조 (Base Weights)	기존 가중치를 미세하게 수정
비용	매우 비쌌 (수개월, 수천 대 GPU) 범용적인 목적	상대적으로 저렴 (몇 시간~며칠) 특정 작업을 목적

3. 비지도 학습 (Unsupervised): 정답 없이 데이터만 줍니다. 비슷한 것끼리 **그룹화 (Clustering)**할 때 씁니다. (예: 비슷한 구매 패턴을 가진 고객끼리 묶기)

- 학습 방식: 정답을 맞히지 않고 데이터 안에 숨겨진 패턴이나 구조 자체를 찾음 > 비슷한 것끼리 묶거나 데이터의 핵심 특징만 남김
- 그룹화(Clustering): 고객 세분화, 타겟 마케팅, 추천되는 시스템
- 차원 축소(Dimensional Reduction) 불필요한 정보는 버리고 핵심만 남기기, 데이터 세트의 특징이나 차원의 수를 줄이면서 가장 중요한 정보나 패턴을 보존하는데 사용됨 > , 빅데이터 시각화, 노이즈 제거, 계산 속도 향상 (차원: 데이터에서 하나의 특징이나 변수를 말함)

4. 준지도 학습(Semi-supervised Learning): 지도학습과 비지도 학습을 섞은 하이브리드 방식

- 방식: 소량의 라벨 데이터로 학습 → 대량의 라벨 없는 데이터로 학습하며 가짜 정답 (pseudo-label) 을 붙임 → 진짜 정답 데이터와 가짜 정답 데이터를 합쳐서 다시 모델을 학습 시킴
- 종류:

1. 부정 탐지 (Fraud Identification)

- **상황:** 수억 건의 결제 데이터가 있지만, 전문가가 "이건 사기다!"라고 확정한 데이터(Labeled)는 아주 적습니다.
- **준지도 학습 활용:** 1. 일단 정답 없는 수억 건의 데이터를 분석해서 정상적인 결제 패턴을 배웁니다. → 2. 그 후, 소량의 사기 데이터를 학습시켜 아주 미세한 이상 징후를 잡아냅니다.

2. 감성 분석 (Sentiment Analysis)

- **상황:** 고객의 문의 글이 수만 개인데, 이걸 일일이 긍정/부정으로 나누려면 인건비가 너무 많이 듭니다.
- **준지도 학습 활용:** 1. 대량의 텍스트(Unlabeled)로 언어의 구조를 먼저 익힙니다. → 2. 샘플링된 소량의 라벨링 데이터로 감성선을 학습하여 전체 데이터의 감성을 추론합니다.

3. 문서 분류 (Document Classification)

- **상황:** 회사 내에 수천 개의 보고서, 명세서, 대본이 섞여 있는데 이걸 사람이 다 분류하기엔 너무 많습니다.
- **준지도 학습 활용:** 1. 정답 없는 문서들을 먼저 학습시켜 "이 문서들은 내용이 비슷하네?"라고 그룹을 묶습니다. → 2. 그중 몇 개만 "이건 명세서야"라고 알려주면, 모델이 비슷한 다른 문서들도 자동으로 명세서로 분류합니다.

4. **강화 학습 (Reinforcement):** 잘하면 보상, 못하면 벌점을 주며 시행착오를 통해 배웁니다. (예: 알파고)
 - 구성 요소: 보상 + 행동
 - DeepRacer
- **추론:** 모델 학습이 완료되면 모델이 학습한 정보를 활용하여 예측이나 결정을 내리는 과정
 - 유형:
 - **배치 추론(=일괄 추론, 오프라인 추론):** 컴퓨터가 이미지나 텍스트와 같은 대량의 데이터를 한번에 분석하여 결과를 도출하는 방식, 결과의 정확성이 더 중요함. 비동기식 실행 모드, 가장 비용 효율적임
 - **실시간 추론(실시간 호스팅 서비스):** 컴퓨터가 새로운 정보를 들어오는 즉시, 신속하게 결정을 내려야 하는 상황, 즉시 결정을 내리는 게 더 중요함. 동기식 실행 모드, 대화형
 - **비동기 추론: 최대 1GB까지의 큰 페이로드**를 처리할 수 있고, 실시간 응답이 필요 없을 때 **비용 효율적**입니다.
 - c.f) 배치 추론: 배치추론은 한꺼번에 처리하는 반면, 비동기 추론은 접수는 미리 받고 순서대로 천천히 처리하는 게 차이
 - **서버리스 추론:** 요청이 올 때만 돈을 내며, 워크로드가 간헐적이거나 예측할 수 없는 트래픽일 때 사용함. 단점은 콜드 스타트(처음 출발할 때 예열 시간이 오래 걸린다는 뜻)
- 성격: 확률적이면서 결정론적 > 확률을 계산한뒤 최종적으로 이진 **라 결정해주기 때문
- 인프라 - AWS 전용 머신러닝 칩
 - **AWS Trainium (학습용):**
 - **개념:** 1,000억 개 이상의 파라미터를 가진 거대 모델의 **딥러닝 학습(Training)**을 위해 AWS에서 특별히 제작한 고성능 칩입니다.
 - **특징:** NVIDIA GPU 대비 **낮은 비용**으로 고성능 학습이 가능하며, EC2 **Trn1** 인스턴스에서 사용됩니다.
 - **키워드:** 모델 학습(Training), Trn1 인스턴스, 고성능/저비용.
 - **AWS Inferentia (추론용):**
 - **개념:** 학습된 모델을 실제로 실행하여 결과를 내놓는 **추론(Inference)** 과정에 최적화된 저비용, 고성능 칩입니다.
 - **특징:** 실시간 생성형 AI 서비스나 딥러닝 앱을 운영할 때 **가장 낮은 비용**으로 추론할 수 있게 설계되었으며, EC2 **Inf1/Inf2** 인스턴스에서 사용됩니다.
 - **키워드:** 모델 추론(Inference), Inf 인스턴스, 낮은 지연 시간(Latency).
- **데이터 전처리 알고리즘**

학습을 시작하기 전, 컴퓨터가 읽기 좋은 상태로 데이터를 다듬는 단계입니다.

(알고리즘은 특정 단계에 고정된 것이 아니라, 데이터를 변환하거나(준비), 정답을 맞히거나(해결), 결과를 채점하는(심사) 모든 과정에서 각기 다른 목적으로 쓰임)

- 데이터 유형: 데이터 유형에 따라 처리 방법도 달라짐 **1. 정형 데이터 (Structured Data): "표(Excel) 형태", 칸 안에 들어가 있음** 이미 규격이 정해진 데이터입니다. 하지만 모델이 더 잘 이해하도록 '다듬는' 과정이 필요합니다.

- **정규화 (Normalization):** 숫자들의 범위를 맞추는 것.
 - 예: 나이(0~100)와 연봉(0~1억)을 똑같이 0.0 ~ 1.0 사이의 숫자로 변환합니다. (안 그러면 연봉 숫자가 너무 커서 모델이 나이를 무시해버려요!)
- **결측치 처리 (Handling Missing Values):** 데이터가 비어있을 때 평균값으로 채우거나, 그 행을 삭제하는 작업입니다.
- **범주형 인코딩 (Encoding Categorical Variables):** '빨강, 파랑' 같은 글자를 '0, 1' 같은 숫자로 바꾸는 작업입니다.

- 1. **비정형 데이터 (Unstructured Data): "텍스트, 이미지", 칸 안에 넣기 힘든 데이터** 규격이 없는 데이터입니다. 모델이 읽을 수 있게 '구조'를 새로 만들어야 합니다.

- **토큰화 (Tokenization):** 문장을 단어나 형태소 단위(Token)로 쪼개는 첫 단계입니다.
- **벡터화 (Vectorization):** 쪼개진 단어들을 고차원의 숫자 배열(Vector)로 바꿉니다.
 - 비유: '사과'라는 단어를 [0.1, -0.5, 0.9...] 처럼 좌표 공간 위의 점으로 표시
- **특징 추출 (Feature Extraction):** 이미지에서 선, 면, 색상 등의 특징을 뽑아내어 숫자로 수치화합니다.

- 알고리즘 유형:

1. 데이터 압축형

- **PCA (Principal Component Analysis)**
 - **한글명:** 주성분 분석
 - **의미:** 데이터에서 가장 중요한 성분(**Principal Component**, 즉 분산이 가장 큰 방향)을 찾아내어 분석(**Analysis**)한다는 뜻입니다.
 - **용도:** 불필요한 변수를 쳐내고 핵심만 남길 때 씁니다.
- **SVD (Singular Value Decomposition)**
 - **한글명:** 특잇값 분해
 - **의미:** 행렬을 특이한 값(**Singular Value**)들을 가진 세 개의 행렬로 쪼개어 (**Decomposition**) 분해한다는 수학적 용어입니다.
 - **용도:** 복잡한 데이터를 단순화하거나 노이즈를 제거할 때 씁니다.

1. 단어 박제형

• Word2Vec (Word to Vector)

- **의미:** 말 그대로 단어(**Word**)를 숫자 배열인 벡터(**Vector**)로(2 = to) 바꾼다는 뜻입니다.
- **특징:** 단어를 고정된 좌표(Vector)에 점 찍듯이 저장합니다. 한 번 점이 찍히면 (학습되면) 주변 문맥이 바뀌어도 그 위치는 변하지 않습니다.

1. 문맥 파악형 BERT (Bidirectional Encoder Representations from Transformers)

- 의미:
 - **Bidirectional**: 양방향. 단어를 읽을 때 앞뒤를 동시에 다 본다는 뜻입니다.
 - **Encoder**: 입력된 정보를 핵심 특징으로 변환하는 장치입니다.
 - **Representations**: 데이터를 표현(수치화)한 결과물입니다.
 - **Transformers**: 구글이 만든 혁신적인 인공지능 모델 구조 이름입니다.
- 특징: 문맥을 완전히 이해한 상태에서 단어의 숫자값(Embedding)을 그때그때 다르게 생성합니다.

3. 딥러닝 (DL: Deep Learning)

- 정의: 인간의 뇌 구조를 본뜬 **인공신경망(Neural Networks)**을 사용하여 복잡한 패턴을 학습하는 ML의 하위 분야
- 구조: 신경망은 '노드'라고 불리는 수많은 작은 단위들이 연결되어 있으며, 이 노드들은 각 입력층, 하나 이상의 은닉층(Hidden Layer), 그리고 출력층으로 이루어져 있음 → 신경망은 노드 간의 연결을 조정하여 패턴을 식별하는 방법을 학습함
- 신경망 종류: NN, Neural Network
 - **CNN(Convolutional NN)**: 합성곱 이미지나 영상 처리에 특화, 공간적인 특징을 분석함
 - **RNN(Recurrent NN)**: 순서 특화, 과거의 정보를 기억해 다음을 예측함 - 텍스트 분석, 주가 예측, 번역기, ...
- 특징: 층(Layer)이 아주 깊고 많아서 'Deep'이라는 이름이 붙었습니다. 이미지 인식, 음성 번역처럼 아주 복잡한 데이터 처리에 강합니다.
- 필수 장비: 연산량이 엄청나기 때문에 고성능 **GPU**가 필수
- 학습 과정:
 1. 손실 함수(Loss Function): 정답 대비 오차를 측정
 2. 경사 하강법(Gradient Descent): "어느 방향으로 조준을 수정해야 오차가 줄어들까?" 어느 방향으로 내려가야 타겟에 가장 빠르게 도착할 수 있을지 발 밑의 경사를 확인하는 과정
 3. 가중치 업데이트: 오차가 난 만큼 가중치를 수정해서 다음번에는 오차를 더 줄여나가는 과정, 학습률은 얼마나 크게 움직일 것인가를 결정하는 상수값으로 적절한 **학습률**을 곱해서 조준(가중치)을 '**적당히**' 조금씩 수정해 나가는 것

4. 생성형 AI (Generative AI)

- 정의: 인터넷 규모의 데이터로 사전 학습된 모델인 기초 모델(Foundation Model)을 활용하여 텍스트 생성, 요약, 정보 추출, 이미지 생성, 질문 답변 등 다양한 작업을 수행함
- 생명주기:

1. 사용 사례 정의 → 2. FM 선택 → 3. 성능 향상 → 4. 결과 평가 → 5. 배포

- 아키텍처 및 작동원리: 모델이 데이터를 어떻게 학습하고 생성하는지에 대한 기술적 뼈대

- 최신 FM 모델의 아키텍처

- **Transformer**: 문맥과 관계를 파악 (현재 LLM의 핵심) > "어텐션"
- **Diffusion (확산)**: 노이즈를 제거하며 생성 (현재 이미지 FM의 핵심)
 - 목적: 잡음이 섞인 데이터에서 고품질 샘플을 생성하는 것이 목적
 - 활용: 고품질의 이미지 생성, Stable Diffusion
 - 모델: Stable Diffusion, Text to Image가 주전공임!

- 전통 생성 기법(이미지 생성 관련)

- **Generative Adversarial Network(GAN)**: 두 모델(생성자, 판별자)의 경쟁을 통해 생성 → 생성형 인공지능의 대표적인 예
 - 목표: 실제 데이터와 유사한 새로운 데이터를 생성하기 위해
 - 활용: 합성 데이터 생성 시 사용, 통계적 특성을 유지해야 할 때
- **Variational Autoencoder(VAE)**: 데이터를 압축(인코더)하고 복원(디코더)하며 생성, 기존 데이터와 유사한 새로운 데이터를 생성하는 게 목적. 학습된 분포에서 샘플링하여 데이터를 생성
 - 활용: 특징 추출, 데이터의 효율적인 압축

- 기초 모델(Foundation Model)

- 정의: 아주 방대한 데이터로 **사전 학습(Pre-training)**되어, 하나만 가지고도 요약, 번역, 코딩 등 **수만 가지 일**을 다 할 수 있는 '범용 모델'입니다.
- 특징: 특정 작업이 아니라 **범용적인 능력**을 가진 모든 거대 모델을 통칭합니다
- 생명 주기: 데이터 선택 > 사전 훈련 > 최적화(핵심 단계) > 평가 > 배포 > 피드백
- 기준: cost, modality, latency, multi-lingual support, model size, ...
- 종류: 그 뼈대를 가지고 어떤 데이터를 처리하느냐에 따라 분류
 - **대규모 언어 모델(LLM)**: 토큰, 임베딩 및 벡터를 사용하여 텍스트를 이해하고 생성함. 언어의 복잡한 관계까지 포착할 수 있어 일관성 있고 문맥에 맞는 텍스트를 생성함
 - **특징**: FM 중에서도 특히 '**인간의 언어(Text)**'를 처리하는 데 특화된 모델입니다
 - **분야**: 요약, 번역, 대화, 텍스트 분류 등 **언어 관련 작업**만 집중합니다
 - **개념**:
 - **토큰**: 모델이 처리하는 텍스트의 기본 단위, 입력 데이터의 표준화를 제공하며 비용 청구의 기준이 됨(토큰 수를 줄이면 비용 절감)
 - **임베딩**: 토큰을 수치적으로 표현한 것으로 각 토큰에는 의미와 다른 토큰과의 관계를 나타내는 벡터(숫자 목록)가 할당됨
 - **컨텍스트 윈도우**: 모델이 입력 받았을 때, 한번에 고려할 수 있는 최대 토큰의 수. 이걸 넘어가면 모델이 대화의 앞부분을 까먹기 시작하며 이게 클수록 비용이 비싸지고 속도가 느려짐

- **확산 모델(Diffusion)** : 순수한 노이즈 또는 무작위 데이터에서 시작하는 딥러닝 아키텍처 시스템. 점진적으로 노이즈에 의미있는 정보를 추가하여 이미지나 텍스트와 같은 명확하고 일관된 출력을 생성함 (예. 텍스트로 이미지를 변환하는 모델)
 - 과정: 초기 데이터 샘플에 제어된 무작위 변화를 반복적으로 돌림
 - **순방향 확산(=전방 확산)**: 입력 이미지에 소량의 노이즈를 점진적으로 도입하여 노이즈만 남을 때까지 진행함
 - **역방향 확산**: 잡음이 섞인 이미지가 점진적으로 잡음 과정에 도입되어 새로운 이미지가 생성됨
- **멀티 모달 모델**: 여러 유형의 데이터를 동시에 처리하고 생성할 수 있으며, 비디오 자막 자동화, 텍스트와 시각 정보를 결합한 질문 답하기 등에 활용됨
 - 방식:
 - 멀티 모달 생성 모델: 새로운 콘텐츠 생성, 대화하기 설명하기 창작하기에 적합
 - 멀티 모달 임베딩 모델: 데이터의 수치화, 검색하거나 유사도 비교, 분류하기에 적합, 생성모달 저렴함

• 모델 출력 최적화

○ 미세 조정(Fine Tuning) >> 지도학습, Labeled Data

- **정의**: 사전 훈련된 모델(FM)에 **레이블이 지정된(Labeled)** 특정 목적의 데이터를 추가하여 **가중치(W)**를 직접 수정하는 지도 학습 과정입니다.
 - 레이블이 지정된 데이터: 입력 특징과 그에 연결된 출력 레이블이 포함되어 있음
 - 레이블이 지정되어 있지 않은 데이터: 입력 특징만으로 구성되며, 대응하는 출력 레이블이 없음
 - c.f)
 - **Pre-training**: 인터넷의 방대한 데이터를 정답 없이 스스로 학습 (비용 매우 비쌈)→ 자기지도학습
 - **Fine-tuning**: 소량의 정답지(Label)를 가지고 특정 목적에 맞춰 교정 (비용 저렴)
- **특징**: 모델의 '뇌' 자체가 바뀌는 과정이며, 특정 도메인의 지식 습득보다는 **특정한 말투, 형식, 전문적인 작업 수행 능력**을 갖추는 데 유리합니다.

○ 프롬프트 조정: 특정 명령어에 어떻게 반응해야 하는지에 대한 예시를 활용함

- 예) 사용자의 연령에 따라 다른 답변 출력, 금지어 지정 등

○ 인간 피드백 기반 강화 학습(RLHF)

- **정의**: 인간의 선호도가 반영된 데이터를 통해 모델이 **더 인간답고 도움이 되는 답변**을 하도록 미세 조정하는 기법입니다.
- **특징**: 유해한 답변을 거르거나, 모델의 답변 품질을 '인간의 기준'에 맞게 정렬(Alignment)할 때 사용합니다.

○ 검색 증강 생성(RAG)

- **기술**: 도메인 관련 데이터를 컨텍스트로 제공하여 해당 데이터를 기반으로 응답을 생성하는 기술임. 모델이 미리 배우지 못한 **최신 정보나 회사의 내부**

기밀 데이터를 실시간으로 '찾아서(Retrieval)' 답변에 참고하게 만드는 기술입니다.

- **정의:** 모델을 새로 학습시키지 않고, **외부의 지식 저장소(PDF, 데이터베이스 등)**에서 질문과 관련된 내용을 **검색(Retrieval)**하여 그 내용을 바탕으로 답변을 **생성(Generation)**하는 기술입니다. >> **이건 Customization이 아님**
- **특징: 비용 효율성, 최신성, 신뢰성, 투명성**
 - **비용 효율성 (Cost-Effective):** 모델 재학습(Fine-tuning)에 필요한 비싼 GPU 비용이 들지 않습니다.
 - **최신성 (Up-to-date):** 새로운 정보(예: 최신 카탈로그)가 생기면 데이터베이스에 파일을 업로드하기만 하면 즉시 반영됩니다.
 - **신뢰성 (Grounding):** 답변의 근거가 되는 문서를 직접 참조하므로, AI가 거짓말을 하는 **환각(Hallucination) 현상**을 획기적으로 줄입니다.
 - **투명성:** 답변과 함께 "어떤 문서의 몇 페이지를 참고했는지" 출처 (Source)를 제시할 수 있습니다.
- Case: 고객 서비스 챗봇(매뉴얼 검색), 의료/법률 지식 조회 등 정답이 어딘가 이미 기록되어 있는 경우에 적합함 (독창적으로 생성해야 하는 경우 적합하지 않음)
- C.F) 미세 조정은 소수의 레이블이 지정된 예제로 파인 튜닝을 하고 기본 모델의 가중치를 변경하는 반면, RAG는 관련 문서들을 검색하여 이를 컨텍스트로 활용하며 기본 모델의 가중치를 변경하지 않음

구분	Fine-tuning (파인튜닝)	RAG (검색 증강 생성)
모델 가중치(W)	변경됨 (공부해서 뇌가 바뀜)	변경 안 됨 (사전 찾아보기)
주요 목적	특정한 말투, 전문 분야 학습	최신 정보, 외부 지식 활용
업데이트 주기	가끔 (큰 변화가 있을 때)	수시로 (데이터가 바뀔 때마다)
비용	비쌈 (GPU 연산 필요)	상대적으로 저렴함

- **차이점 (핵심!):**
 - **전통적 AI:** 데이터를 분석해서 "이건 스팸 메일이다/아니다"를 판단 (**Discriminative**)합니다.
 - **생성형 AI:** "스팸 메일 예시를 하나 써줘"라고 하면 **생성(Generative)**합니다.
- **기반 모델: Foundation Models(FM)나 LLM(거대 언어 모델)**이 여기에 해당하며, AWS에서는 **Amazon Bedrock**을 통해 이들을 제공합니다.
- **사업 지표:** 사용자 만족도, Average Revenue per user, Cross-domain performance, Conversion rate, Efficiency,
- **Capability:** Personalization, Scalability, Simplicity, Responsiveness

구분	RAG (Knowledge Bases)	Fine-tuning (파인 튜닝)
비유	오픈북 테스트 (책 보고 답하기)	장기 기억 개조 (머릿속 지식 수정)
목적	최신 정보/외부 데이터 활용	특정 말투/형식/전문 도메인 습득
강점	사실 정보의 정확성 (Fact-checking)	모델의 행동 양식 및 스타일 변경

구분	RAG (Knowledge Bases)	Fine-tuning (파인 튜닝)
업데이트	매우 쉬움 (문서만 바꾸면 됨)	어려움 (다시 학습시켜야 함)

[제2장: 생성형 AI 핵심 기술]

여기서는 AI가 어떻게 똑똑하게 대답하는지 그 '방식'을 배웁니다.

- **핵심 아키텍처: Transformer**, 현대 AI의 '뇌 구조'입니다. 문장에서 단어 간의 관계를 파악하는 **Attention(집중)** 매커니즘이 핵심입니다.
- **모델 종류:**
 - **LLM (언어)**: 토큰(단위), 임베딩(수치화), 벡터(의미 공간)를 사용
 - **Diffusion (이미지)**: 노이즈를 추가(순방향)하고 제거(역방향)하며 이미지 생성
 - 모델 예시) Stable Diffusion(Stability AI사), Text to Image가 주특기!
 - **Multi-modal**: 텍스트, 이미지, 오디오 등 여러 유형을 동시에 처리
 - 모델 예시) TITAN(Amazon 사)
- **주요 현상: 이를 해결하기 위해 RAG 사용**
 - **Hallucination(환각)**: AI가 모르는 걸 아는 척 거짓말하는 현상입니다.
 - **Nondeterminism(비결정성)**: 동일 입력 데이터를 사용하더라도 모델이 실행될 때마다 다른 출력을 생성하는 문제 >> 이게 LLM의 특징이기도 함
- **RAG (검색 증강 생성): (시험 단골)** AI에게 최신 백과사전(S3 등 외부 데이터)을 쥐여주고 "여기서 찾아서 대답해!"라고 시키는 기술입니다. 환각을 줄이는 최고의 방법이죠.
 - 단계:
 1. **Retrieve (검색)**: 질문과 관련된 문서를 DB(주로 Vector DB)에서 찾음
 2. **Augment (증강)**: 찾은 문서를 프롬프트에 끼워넣어(Augment) 답변을 보강함
 3. **Generate (생성)**: 합쳐진 정보를 바탕으로 AI가 대답함
 - **장점**: 모델을 새로 학습시키지 않아도 최신 정보를 반영할 수 있고 비용이 저렴함
 - 엔진: **Amazon OpenSearch Service (RAG의 엔진)**
 1. 왜 쓰는가? (The "Why")
 - 일반 데이터베이스(RDS 등)는 "이 단어가 있니?"라고 묻는 **키워드 매칭**에 강해요.
 - 반면, OpenSearch는 "이 문장이랑 의미가 가장 비슷한 게 뭐야?"라는 유사도 점수(Similarity Scoring) 계산에 특화되어 있습니다.
- **2. RAG에서의 결정적 역할**
 - **Full-text Search**: 단어 그대로를 찾는 검색 (예: "아이폰 15")

- **Vector Search:** 단어의 '의미'를 숫자로 바꿔서 찾는 검색 (예: "최신 애플 스마트폰"이라고 쳐도 "아이폰 15"를 찾아냄)
 - **Ranking:** 찾은 문서들 중 어떤 게 질문과 가장 관련 있는지 점수를 매겨서 줄을 세워줍니다.
- **Negative Prompting:** 결과물에 포함하지 않아야 할 내용을 미리 알려줌
- **Feature Engineering:** 원본 데이터(Raw Data)를 머신러닝 알고리즘이 더 잘 이해하고 학습할 수 있는 형태(Future)로 변환, 선택, 수정하거나 새로운 특징을 만들어내는 과정입니다. 기존 데이터를 가공해서 새로운 특성을 만들 → 데이터 전처리 단계
 - 작업:
 - 특징 생성: 기존 데이터를 조합해 새로운 컬럼 만들기
 - 특징 변환: 숫자의 범위를 맞추거나 로그를 취하기
 - 특징 선택: 기존 특징 중 중요한 것만 골라내기, 해석력을 높임
 - 특징 추출: 데이터를 압축해 새로운 축으로 변환, 연산 속도를 높임
 - 예) 딸기 + 설탕 > 딸기 시럽을 아예 만들어버림, 전혀 다른 특징을 만들어 낸 것
 - 특징:
 - **모델 성능 향상:** 단순히 데이터를 많이 넣는 것보다, 잘 정제된 피쳐 하나가 모델의 **정확도(Accuracy)**를 획기적으로 높입니다.
 - **컴퓨팅 효율성:** 불필요한 데이터를 제거하고 핵심 피쳐만 남김으로써 학습 속도를 높이고 메모리 사용량을 줄이는 **효율성**을 제공합니다.
 - **도메인 지식 활용:** 해당 산업(금융, 의료 등)의 지식을 활용해 "어떤 데이터가 정답에 가장 큰 영향을 줄까?"를 고민하여 새로운 피쳐를 창조합니다.
- **Prompt Engineering:** 입력의 기술, 모델에게 주는 **명령어(텍스트)** 자체를 수정함
 - 구성 요소: 지침(Instruction, 수행해야 할 작업), 맥락(모델을 안내하는 외부정보), 입력 데이터, 출력 데이터
- **모델 파라미터:** 모델과 입력 해석 및 응답 생성 과정에서 모델 동작을 정의하는 값. 학습 과정에서 학습되고 조정되는 모델의 내부 변수 모델의 지능 그 자체. 입력값이 들어왔을 때 답을 내는 핵심 로직, 학습 중에 계속 업데이트
 - 종류: 가중치, 편향 등
 1. 가중치: 어떤 특징(Future)이 정답과 더 밀접한 관련이 있는지 결정합니다.
 2. 편향: 모든 입력값이 0이어도 결과가 0이 되지 않도록 하거나, 모델의 판단 기준(결정 경계)을 통째로 이동시켜 데이터에 더 잘 맞게 조절합니다.
- **HyperParameter:** 모델을 맞춤 설정하기 위해 조정하는 값, 학습 과정을 제어하고 결과적으로 출력되는 사용자 지정 모델을 결정함. 학습 단계에서 모델을 잘 가르치지 위해 사용함, 하이퍼파라미터
 - 종류: 학습률, 배치 사이즈, 에포크 등 ('하'이퍼 파라미터니까 '학습 방법'에 대한 것)

1. 에포크(Epoch)

- 인공지능 모델이 준비된 전체 학습 데이터를 딱 한번 다 훑었을 때 말하는 단위
- 보통 여러번 돌려서 데이터 속에 숨겨진 패턴과 관계를 더 학습시키는데 적당한 지점에서 멈추는 게 중요

2. **Learning Rate:** 한 번 공부할 때 얼마나 큰 폭으로 배울 것인가? (너무 크면 성적이 튀고, 너무 작으면 공부가 안 끝남)

3. **Batch Size:** 한 번에 몇 문제씩 풀고 채점할 것인가?

구분	하이퍼파라미터 (Hyperparameter)	모델 파라미터 (Model Parameter)
결정 시점	학습 시작 전 (Pre-set)	학습 과정 중 (Learned)
조정 주체	사람(개발자)이 직접 손으로 입력	알고리즘(컴퓨터)이 데이터를 보고 스스로 조정
역할	학습의 규칙과 속도를 제어	입력 데이터에 대한 정답을 도출
성격	외부 구성 (Configuration)	내부 변수 (Internal variables)
예시	학습률(Learning Rate), 레이어 수, 에포크(Epoch) 수	가중치(Weights), 편향(Bias)

- **추론 매개변수:** 추론 단계에서 사용, 설정의 기술, 모델이 답변을 생성하는 **내부 알고리즘의 동작 방식**을 조정함

- 특징: 모델의 답변 생성 방식을 제어하며, 이미 학습이 끝난 모델을 호출할 때 결정함
- 종류

1. Top-K (가장 가능성 있는 'K'개의 후보, Kount)

- **핵심:** 다음에 올 단어 중 확률이 높은 순서대로 **딱 K개**만 골라놓고 그 안에서 선택합니다.
- **사용자님의 문제 상황:** "가장 가능성이 높은 후보의 **개수(수)**를 조절하고 싶다" → **Top-K**가 정답!
- **예시:** Top-K가 50이면, 상위 50개 단어만 후보로 두고 나머지는 버립니다.

2. Top-P (확률의 '합계'가 P가 될 때까지의 후보, Probability)

- **핵심:** 개수가 아니라 **확률의 누적 합계**를 기준으로 합니다.
- **특징:** 확률이 높은 단어가 많으면 후보 수가 적어지고, 확신이 없으면 후보 수가 늘어납니다. (다이내믹함)
- **창의성 영향:** top-p 값을 낮추면 모델이 더욱 보수적으로 변하여 가장 가능성이 높은 단어만 선택하게 되므로 창의성이 저하됩니다. 반대로 top-p 값을 높이면 모델이 더 광범위한 단어를 고려하게 되어 더욱 다양하고 예측 불가능한 언어를 사용하게 되므로 창의성이 향상됩니다.

- **키워드:** 누적 확률, 합계, 가변적인 후보 수
- 예시: p값을 조절한다, 합계가 90% 될 때까지 가져와" (개수는 변함)

3. Temperature (확률 분포의 '변동성/창의성')

- **핵심:** 확률 분포 자체를 깎거나 부풀려서 **창의성**을 조절합니다.
- **특징:** 낮으면(0에 가까우면) 가장 빠른 답만 하고, 높으면(1에 가까우면) 엉뚱하고 창의적인 답을 합니다. 1이 가까워지면 환각 현상이 발생할 확률이 높음 (온도가 낮으면 뽀뽀 얼어서 빠른 것만 대답함..ㅎ 높으면 자유로운 기체 상태..ㅎ)
- **키워드:** 창의성, 무작위성, 결정론적 답변

구분	모델 파라미터 (Internal)	하이퍼파라미터 (External)	추론 매개변수 (Runtime)
핵심 개념	모델 내부의 '지식'	모델의 '학습 방법'	모델의 '응답 스타일'
결정 시점	학습 과정 중에 결정됨	학습 시작 전에 결정함	학습 완료 후 추론(사용) 시 결정
조정 주체	알고리즘 (컴퓨터)	개발자 (사람)	사용자 (API 호출자)
변경 가능성	고정됨 (재학습 전까지)	고정됨 (재학습 전까지)	매 호출마다 자유롭게 변경 가능
주요 예시	가중치(Weights), 편향 (Bias)	학습률(Learning Rate), 배치 크기	Temperature, Top-P, Max Tokens

○ Regulation, 정규화

- 정의: 모델이 공부할 때 너무 복잡한 공식을 쓰지 말라고 벌금을 매기는 것
- 특징: 정규화를 높이면 모델이 아주 단순한 법칙만 찾으려고 함 > Overfitting 해결 | 정규화를 낮추면 모델에게 자유를 주어서 복잡한 공식을 만들게 됨 > 훈련 데이터의 정확도가 높아짐

○ 응답 길이

- 정의: 생성된 응답에서 반환해야 하는 최소 또는 최대 토큰 수

○ 정지 시퀀스

- 정의: 모델이 더 이상 토큰을 생성하지 않도록 하는 문자 시퀀스를 지정함

○ Few-shot: 예시를 몇 개 주고 질문 (성능 향상)

- **인식(Recognition)이 목표일 때:** "이 말은 A라는 뜻이야"라는 **라벨, 의 (Intent)**이 필요합니다
- **생성(Generation)이 목표일 때:** "이 말에는 이렇게 답해"라는 **답변 (Response)**이 필요합니다

- **Zero-shot:** 예시 없이 바로 질문, FM이 크고 성능이 좋을수록 효과가 좋음. 데이터셋이 부족하거나 없을 때 가장 효율적인 방법
- **Chain of Thought(CoT):** 복잡한 추론 작업을, 더 작고 중간 단계들로 나누는 기법

[제3장: 모델 평가]

학습된 모델이 얼마나 똑똑한지 점수를 매기고 비교하는 단계입니다.

** 모델 학습

1. 데이터 분할: 데이터셋은 **훈련 세트** 와 **검증 세트**로 나뉩니다.

—> 나누는 거 자체는 [모델 학습] 단계임, 데이터 수집 단계 아님 주의!

2. 매개변수 조정 및 미세 조정: 하이퍼파라미터(학습률, 배치 크기, 정규화 계수 등)는 모델 성능을 최적화하기 위해 훈련 과정에서 미세 조정됩니다.

3. 모델 학습

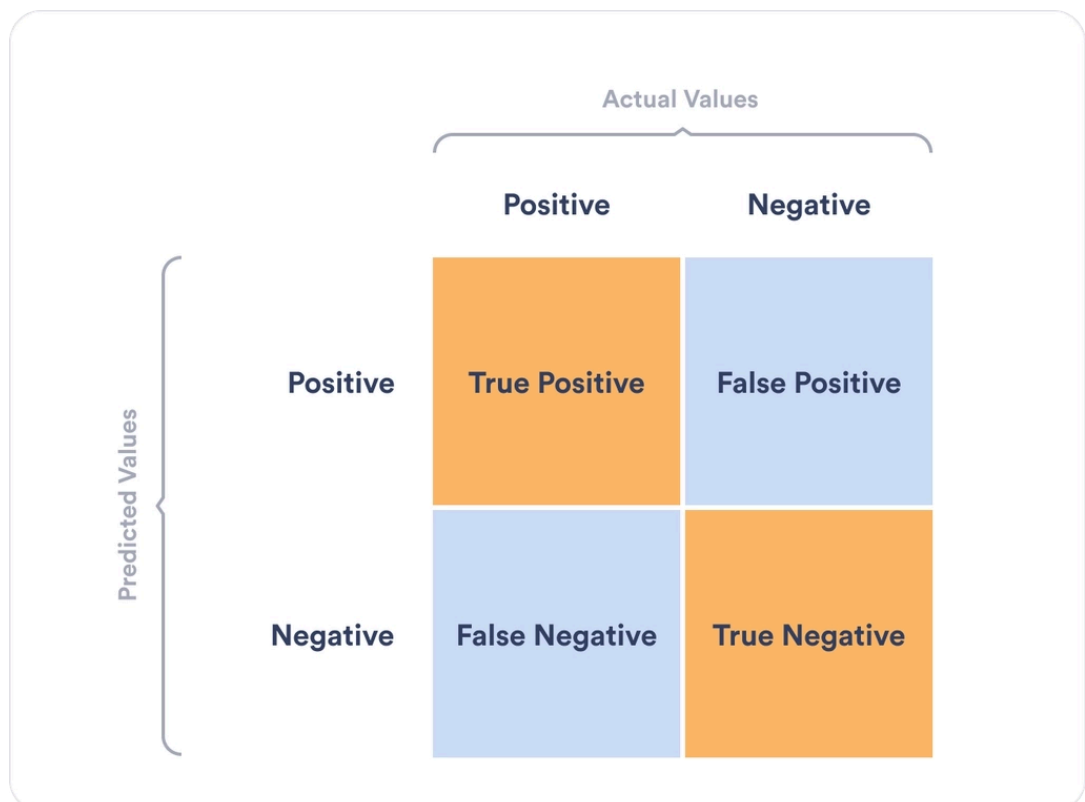
- 모델 평가 데이터셋:

- **Validation Set(선택):** 모델이 실제 환경에서 어떻게 반응하는지 평가하기 위한 별도의 데이터 세트
 - **역할:** 학습 중인 모델의 성능을 중간 점검하고, **하이퍼파라미터(모델의 설정값)**를 최적으로 조정(Tuning)하는 데 사용됩니다. > 가중치 업데이트 하지 않음!
 - c.f) 훈련 세트에서만 가중치 업데이트를 함
 - **특징:** "공부가 잘되고 있나?"를 확인하는 용도입니다. 모델이 너무 복잡해지거나 단순해지지 않게 가이드를 줍니다. (**선택 사항/Optional**)
 - **조기 종료 기능:** 검증 성능이 더 이상 향상되지 않을 때 학습을 중단
- **Test Set(필수):** 검증 데이터를 사용하여 모델을 개선한 후, 모델의 최종 성능 - 예측 정확도가 '기준'을 충족하는지 확인하기 위한 데이터 세트
 - **역할:** 모든 학습이 끝난 모델이 **처음 보는 데이터(Unseen Data)**에서 얼마나 잘 작동하는지 **최종 성능**을 평가합니다.
 - **특징:** 이 데이터는 학습이나 튜닝에 절대 관여하지 않습니다. 오직 '일반화 (Generalization)' 능력을 확인하는 용도입니다. (**필수**)
- **Benchmark Dataset:** 업계 전체가 약속한 표준화된 공개 데이터로 가성비 최고의 평가 수단임
 - **특징:** 최소한의 노력, 표준화된 검증, 비용 효율성 최소한의 관리 노력으로 LLM의 편향 및 차별 여부를 평가하는 데 가장 적합한 옵션입니다. 이러한 데이터셋은 모델 출력의 잠재적 편향을 테스트하는 다양한 시나리오를 포함하도록 특별히 설계 및 구성되었습니다.
 - **예시:** 자연어 처리 작업에 사용

- GLUE(General Language Understanding Evaluation): 텍스트 분류, 질의응답, 자연어 추론과 같은 언어 이해 작업을 평가하기 위한 데이터셋 모음
- SuperGLUE: GLUE의 확장 버전으로, 더욱 어려운 과제를 수행하고 구성 언어 이해에 중점
- 스탠포드 질의응답 데이터셋(SQuAD): 질의응답 기능을 평가하기 위한 데이터셋
- 기계 번역 워크숍(WMT): 기계 번역 시스템을 평가하기 위한 일련의 데이터 세트 및 작업

• 혼동 행렬(Confusion Matrix)

- 정의: 분류 모델이 예측한 값과 실제 값이 얼마나 일치(혹은 혼동)하는지 보여주는 표로, 단순 정확도(Accuracy)만으로는 알 수 없는 '어디서 틀렸는가'를 세밀하게 분석하는 도구
- 지표:
- **True Positive (TP)**: 맞는 걸 맞다고 잘 맞춤
- **True Negative (TN)**: 틀린 걸 틀리다고 잘 맞춤
- **False Positive (FP)**: 틀린 건데 맞다고 오해함 (1종 오류)
- **False Negative (FN)**: 맞는 건데 틀리다고 놓침 (2종 오류)



- 자동화된 측정 지표 : 빠르고 비용이 싸지만, 맥락을 읽지 못한다는 단점이 있음. 정량적임

1. 언어 모델(LLM) 평가 지표

텍스트를 생성하는 모델은 주로 '기계적인 유사도'와 '추론 능력'을 봅니다.

- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation): 재현율 (놓치지 않는 능력)에 중점**
 - **용도: 요약(Summarization)** 성능 측정, 정확한 단어 일치 기반
 - **방식:** 모델이 만든 요약문이 사람이 쓴 정답 요약문과 얼마나 겹치는지 (단어의 중복도)를 봅니다.
 - **사용방법:**
 - **ROUGE-N:** 생성된 텍스트와 참조 텍스트 간의 n-그램 중복 정도를 측정합니다. 예를 들어, ROUGE-1은 유니그램, ROUGE-2는 바이그램 등의 중복을 나타냅니다. 이 지표는 주로 텍스트의 유창성과 참조 텍스트의 핵심 아이디어를 얼마나 잘 포함하고 있는지를 평가합니다.
 - **ROUGE-L:** 생성된 텍스트와 참조 텍스트 간의 가장 긴 공통 부분 수열을 사용합니다. 특히 출력물의 서술의 일관성과 순서를 평가하는 데 효과적입니다.
 - **BLEU (Bilingual Evaluation Understudy): 근본적으로 정밀도를 측정하는 지표**
 - **용도: 번역(Translation)** 성능 측정
 - **방식:** 모델의 번역 결과와 전문가의 번역본이 얼마나 일치하는지 계산합니다. 1에 가까울수록 좋습니다.
 - **Perplexity (PPL, 혼란도):**
 - **방식:** 모델이 다음 단어를 예측할 때 얼마나 헛갈려 하는지를 나타냅니다. 낮을수록 모델이 언어 구조를 잘 이해하고 있다는 뜻입니다. (낮을수록 우수!)
 - **BERT 점수: 의미적 유사성을 평가, 문맥 이해!**
 - 약자: Bidirectional Encoder Representations from Transformers
 - **용도: 텍스트 생성(Text Generation)** 모델의 품질 측정. 특히 요약, 번역, 챗봇 응답처럼 "답변의 의미가 정답과 얼마나 유사한가"를 평가할 때 필수적으로 쓰입니다.
 - **방식: BERT 모델을 심사위원으로 활용합니다.** 정답 문장과 생성 문장의 각 단어를 BERT에 넣어 의미가 담긴 **숫자(Embedding)**로 바꾼 뒤, 그 숫자들끼리 수학적으로 얼마나 가까운지(유사도)를 비교해서 점수를 냅니다.(BERTScore는 후보 텍스트와 참조 텍스트에 있는 단어의 문맥 임베딩 간의 코사인 유사도를 계산합니다.)
 - 키워드: 문맥적 의미, 미묘한 뉘앙스
1. 요약 > 요약에서는 '인간 평가'가 강조됨

2. 이미지 생성(Diffusion) 모델 평가 지표

이미지는 "얼마나 진짜 같나"와 "내 명령어를 잘 따랐나"가 중요합니다.

- **FID (Fréchet Inception Distance):**
 - **방식:** 생성된 이미지들의 통계적 분포와 실제 이미지들의 분포를 비교합니다.
 - **특징:** 낮을수록 실제 사진처럼 고퀄리티라는 뜻입니다.

- **IS (Inception Score):**

- **방식:** 생성된 이미지가 얼마나 뚜렷한 물체를 포함하고 있는지(선명도), 그리고 얼마나 다양한 이미지를 만드는지(다양성)를 봅니다. **높을수록** 좋습니다.

- **모델 적합도:** 모델 정확도가 낮은 근본 원인을 파악하는 게 중요

- **Overfitting:** 훈련 데이터에서는 좋으나, 평가 데이터가 안 좋음 (과대적합)

- **원인:** 이미 학습한 데이터를 암기하여 새로운 데이터에 대해서는 일반화 능력을 발휘하지 못하기 때문
- **해결 방법:**
 - 데이터 증강: 데이터를 **살짝 변형해서 데이터의 양을 억지로 늘리는 것**
 - 정규화: 특정 데이터에 과하게 반응하지 않도록 가중치 조절
 - 드롭 아웃: 학습할 때 뉴런을 몇 개씩 무작위로 꺼버려서 편식 학습을 방지
 - 조기 종료: 나빠지기 시작하는 순간 학습 종료
 - 가지치기: 모델의 결정에 큰 영향을 주지 않는 **불필요한 가지(파라미터/피쳐)를 쳐내는 것**
 - 앙상블: 여러 개의 모델(약한 학습기)을 합쳐서 **하나의 강력한 모델**을 만드는 방식입니다.

- **Underfitting:** 훈련 데이터에 대해 제대로 된 성능을 보이지 못하는 현상 (과소적합)

- **원인:** 모델이 입력 데이터와 목표값 사이 관계를 제대로 포착하지 못하기 때문
- **해결 방법:** 복잡도 높이기, 특징 추가, 규제 완화, 학습 시간 및 횟수 늘리기

- **Balanced:** overfitting이나 underfitting이 아닐 때

- **편향과 분산**

- 1. **편향(bias):** 예측값과 실제값의 차이, 데이터 셋의 중요한 특징을 모델이 제대로 반영하지 못하고 있는 경우

- **High Bias:** Underfitting (편향이 높다는 건 모델이 너무 단순하다는 것)
- **해결방법:** 데이터를 더 수집하거나, 부족한 데이터를 복제(Oversampling)하거나, 많은 데이터를 삭제(Undersampling)해서 **재료의 비율**을 맞춰야 합니다
- **종류:**
 - **인간 편향:** 설계자의 주관적 판단이 원인
 - **선택 편향:** 샘플링의 오류
 - **역사적 편향:** 과거 사회의 불평등이 원인 (과거 기록)
 - **알고리즘 편향:** 모델의 수학적 오류 (사람의 개입 X)

- 2. **분산(variance):** 예측값들이 얼마나 퍼져 있는지를 나타냄 , 모델이 훈련 데이터의 변동이나 노이즈에 얼마나 민감한지를 나타냄

- **High Variance:** overfitting (예측값이 휘청휘청 변하는 상태라 너무 복잡함)

- 해결방법: 모델 정규화 - 모델이 너무 특정 데이터에만 '집착(Overfitting)'하지 못하도록 **벌칙(Penalty)**을 주는 거예요. 수학적으로 가중치(W)가 너무 커지지 않게 억제하는 기술입니다.

[정리]

편향과 분산은 동시에 나빠질 수도 있음

High Bias = Underfitting (모델이 너무 멍청해서 규칙을 아예 못 찾음) >> BU

High Variance = Overfitting (모델이 훈련 데이터의 노이즈까지 외워버려서 응용력이 없음) >> VO

높은 편향 = 과소 적합, 화살들이 중앙에서 멀리 쏠려 있음

높은 분산 = 과대 적합, 화살들이 정중앙 근처긴 한데 넓게 퍼져 있음

→ Trade-Off 관계로 하나를 얻으려면 다른 하나를 포기 해야 함

- 모델을 **복잡**하게 만들면? → 정답에는 가까워지지만(Bias 감소), 예민해져서 결과가 들쭉날쭉해집니다(Variance 증가).
- 모델을 **단순**하게 만들면? → 결과는 일정해지지만(Variance 감소), 정답과는 거리가 멀어집니다(Bias 증가)

구분	고편향 (High Bias)	고분산 (High Variance)
상태	과소적합 (Underfitting)	과대적합 (Overfitting)
원인	모델이 너무 단순함 (공부 부족)	모델이 너무 복잡함 (기출문제만 달달 외움)
결과	훈련 데이터도 못 맞힘	훈련 데이터는 잘 맞히는데, 새 문제는 틀림
해결책	더 복잡한 모델 사용, 특징(Feature) 추가	데이터 추가, 규제(Regularization) 적용

*정규화(Normalization): 서로 다른 기준을 가진 숫자들을 0~1 사이의 일정한 범위로 맞추는 작업

분류나 회귀나

분류는 정답이 이름(Label)이고, 회귀는 정답이 숫자(Quantity)라 정답의 성격이 다릅니다.

분류는 경계선을 잘 그어서 A와 B를 확실히 가르는 게 중요하고, 회귀는 실제값과 예측값 사이의 오차 거리를 최소로 줄이는 게 중요합니다.

1. 분류 문제 지표: 맞혔나 틀렸나? 혼동 행렬을 기반으로 계산함 (도구: Confusion Matrix)

- **Accuracy:** 정확도, 전체 시도 중 중 정답을 맞힌 비율, '결과가 맞는가 틀리는가?'
- **Precision:** 정밀도, 모델이 예측한 것들 중 실제로 그럴 비율, '자기들끼리 얼마나 모여있는가?'

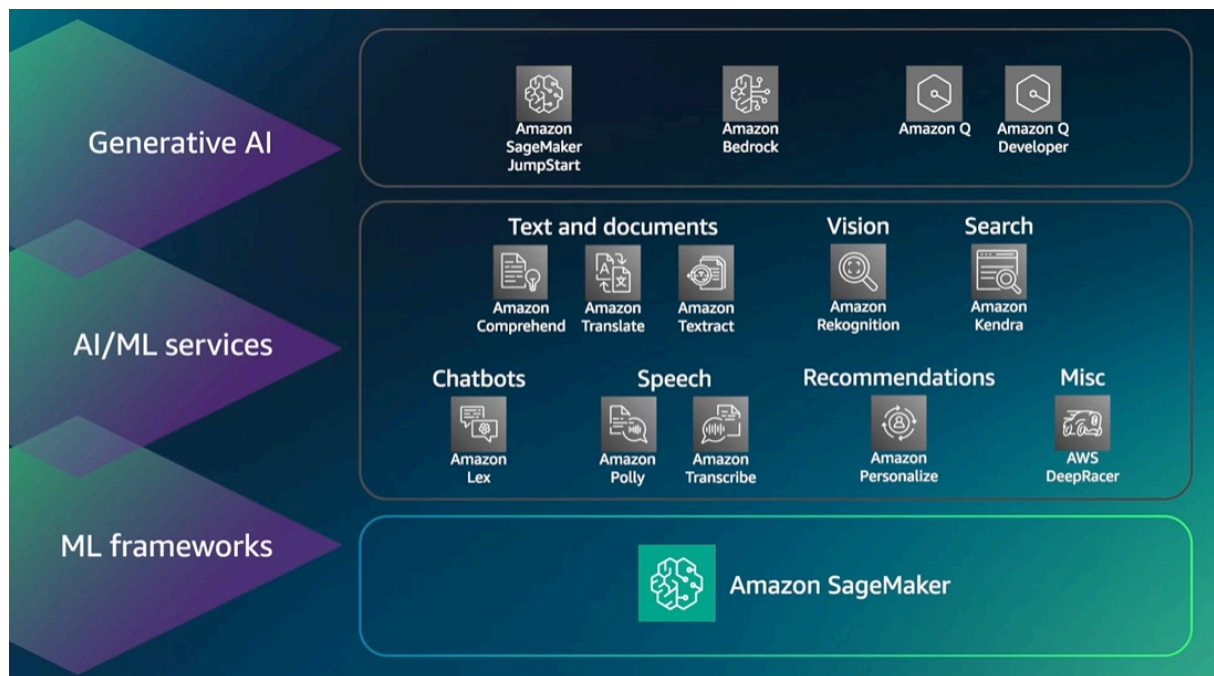
- 틀려도 좋으니 확실할 때만 "정답"이라고 해야 할 때 (예: 스팸 메일 차단 - 멀쩡한 메일을 스팸으로 치면 안 됨)
- **Recall:** 재현율(놓치지 않는 능력), 실제로 positive인 데이터 중에서 모델이 positive라고 맞힌 비율
 - 오답이 섞여도 좋으니 정답을 하나도 놓치면 안 될 때 (예: 암 진단 - 암 환자를 정상이라고 놓치면 큰일 남)
- **F1:** Precision과 Recall의 조화 평균, 두 지표 중 어느 하나가 낮으면 점수가 확 떨어짐 - 두 지표 간 균형을 맞춰줌(동일한 가중치 부여)
- **AUC-ROC(Area Under Curve-receiver Operator Curve):** 모델의 분류 결정 임계치를 변화시킬 때, 모델이 음성과 양성을 얼마나 잘 구별하는지 나타내는 면적 값 (1에 가까울수록 완벽한 구별)

2. 회귀 문제 지표: 얼마나 거리 차이가 나나? (예측값- 정답 간)

- **Mean squared error(MSE):** 평균 제곱 오차, 각 데이터 포인트에서 오차를 제곱하여 모두 더한 뒤 평균을 낸 값, 오차를 제곱하기 때문에 큰 오차에 대해 더 가중치를 크게 둠
- **R Squared:** 결정 계수, 모델이 데이터의 변동성을 얼마나 설명하는지 나타내는 지표(0~1, 1에 가까울수록 모델이 실제 데이터 분포를 완벽하게 설명함)
- **Mean Absolute Error(MAE):** 평균 절대 오차, 예측값과 실제값 차이(오차)에 절대값을 씌워 평균을 낸 것으로 모델이 틀린 양을 측정함
- **Pre-training (사전 학습):** 아무것도 모르는 아기 AI에게 수조 개의 데이터를 먹여 초등학생으로 만드는 과정. (돈/시간 엄청 듬)
- **평가 지표:**
 - **Accuracy:** 얼마나 정확한가?
 - **Perplexity:** AI가 다음 단어를 고를 때 얼마나 헛갈려 하는가? (낮을수록 좋음)

[제4장: AWS 핵심 AI 서비스]

어떤 상황에 어떤 서비스를 써야 하는지 매칭하는 게 시험 문제입니다.



구분	Amazon Bedrock	Amazon SageMaker
대상	이미 만들어진 모델 (FM)	직접 훈련시키는 모델
복잡도	낮음 (말을 잘 듣는지 확인)	매우 높음 (수치와 성능 중심)
목적	빠른 프로토타이핑	정밀한 모델 성능 최적화

1) Amazon Bedrock (The Easy Way)

- **정의:** API를 통해 제공하는 완전 관리형 서비스로 빠르게 시작하고, 실험하며 FM 모델을 원활하게 통합 및 배포할 수 있음
- **특징:** 서버리스, 인프라 구축 안 함. 모든 단계를 지원하긴 하는데 각 단계마다 자유도가 낮음
 - **모델 선택권:** Anthropic, Meta, Mistral 등 여러 회사의 이미 만들어진 모델 중 내가 원하는 것을 고를 수 있습니다.
 - **커스터마이징: Fine-tuning**(미세 조정)이나 **RAG**(지식 기반 연결)를 통해 나만의 전용 모델을 직접 설계합니다.
 - 커스터마이징 방식: 미세 조정, 지속적 사전 학습(라벨링 안된 데이터), 에이전트 및 RAG
 - **데이터 보안:** 커스터마이징에 사용된 데이터는 모델의 기본 학습 데이터로 재사용되지 않으며, 사용자만의 독립된 환경에 암호화되어 보호됩니다.
 - **서버리스(Serverless):** 별도의 GPU 인프라를 구축하거나 관리할 필요 없이, 데이터만 준비하면 AWS가 알아서 학습을 진행합니다. > 수요에 따라 인프라를 자동으로 확장하며 관리할 필요 x
 - 인프라 관리 단순화: 서버 사양, OS 업데이트, 보안 패치 등 다 알아서 해줌
 - 자동 확장: 예측 불가능한 워크로드에 최고~
 - 완전관리형 API 제공: 생성형 AI와 통합하기 위함
- **강점:** 기업 보안 환경에 맞춰 AI 앱을 '**직접 빌드**'하고 확장할 수 있는 환경을 제공합니다.

- c.f) SageMaker: 모델 배포 및 확장까지 해주는 반면 이걸 못함
- 요금제: On-Demand, Provisioned Throughput, Evaluation 밖에 없음 (서버 리스에 api 호출 형태라 스팟 인스턴스 없음)
- **사용자 지정 방법:** Fine Tuning & Pre-training (RAG는 모델의 파라미터나 가중치를 변경하지 않으므로, 사용자 지정은 아님)
- 저장 위치: S3 객체 스토리지 서비스, 기본적으로 BedRock과 통합되어 있으므로 추가 분석을 위해서는 S3에 저장
- 액션 그룹: 에이전트 기능을 완성하는 핵심 부품으로 **외부 시스템과 소통(API 호출)** 하는 구체적인 기능 단위
 - OpenAPI 스키마: "이 API는 어떻게 쓰는 거야?"라고 가르쳐주는 가이드북
 - Lambda 함수: AI가 내린 명령을 받아 실제로 코드를 실행하는 **실제 일꾼**
- 단계별 기능 데이터 준비 단계

- **Knowledge Bases for Amazon Bedrock:** RAG를 구현하기 위한 인프라와 과정을 자동으로 처리해주는 서비스

- 작동 순서:

1. **데이터 소스 연결:** S3 같은 곳에 기업의 비공개 문서(PDF, 텍스트 등)를 올립니다.
2. **텍스트 분할 (Chunking):** 너무 긴 문서는 AI가 읽기 힘들니 적당한 크기로 자릅니다.
3. **벡터화 (Embedding):** 텍스트를 숫자의 나열(Vector)로 바꿉니다. (이때 모델이 의미를 파악하죠!)
4. **벡터 저장소에 저장:** 변환된 숫자들을 전용 데이터베이스(OpenSearch 등)에 차곡차곡 쌓습니다.

- 디폴트: Amazon OpenSearch Serverless (OSS), Bedrock 콘솔에서 Knowledge Base를 만들 때 "Quick create" 옵션을 선택하면, AWS가 사용자를 대신해

자동으로 OpenSearch Serverless 벡터 인덱스를 생성해 줍니다.

- **심리스한 통합:** 권한 설정(IAM)이나 데이터 연결이 AWS 내부망을 통해 가장 매끄럽게 연동됩니다.

모델 평가 단계

- **Amazon Bedrock Playground**

- **개념:** 코드를 짜기 전, AWS 콘솔에서 GUI(화면)를 통해 모델의 성능을 **즉각적으로 테스트**해 볼 수 있는 환경입니다.

- **주요 특징:**

- **다양한 모드:** 텍스트(Text), 채팅(Chat), 이미지(Image) 전용 Playground 제공.
- **매개변수 실험:** Temperature, Top-P 같은 설정값을 바꿔가며 답변이 어떻게 변하는지 확인 가능 실제 애플리케이션에 통합하기 전에 프롬프트와 매개변수를 실험하고 다듬을 수 있도록 특별히 설계된 대화형 환경입니다.

- **프롬프트 엔지니어링**: 어떤 질문(Prompt)을 던져야 가장 좋은 답이 나오는지 미리 연습해 보는 장소.
- **장점**: 개발 초기 단계에서 **어떤 모델이 우리 서비스에 가장 적합한지** 결정하는 데 큰 도움을 줍니다.
- **키워드**: 모델 비교, 추론 매개변수 테스트, 프롬프트 실험.

배포 단계

- **프로비저닝 처리량 (Provisioned Throughput) : 미세조정 모델은 요것만 됨**
 - **개념**: 특정 모델을 위해 **전용 컴퓨팅 용량(통로)**을 미리 예약하는 방식입니다. 나만을 위한 전용 차선을 빌리는 것과 같습니다. > 이미 구매한 용량 단위를 제공함, 안정적인 작업 부하
 - **장점**: 성능 보장, ****다른 사용자의 간섭 없이 일정한 속도(처리량)와 낮은 지연 시간(Latency)을 보장받습니다. > 일관되고 안정적인 성능
 - **커스텀 모델 배포**: 미세 조정(Fine-tuning)된 사용자 지정 모델을 실행하려면 반드시 이 모드가 필요합니다.
 - **한계**:
 - **비용 부담**: 사용량과 상관없이 예약한 시간(보통 1개월 또는 6개월 단위 약정) 동안 비용을 지불해야 하므로 온디맨드(On-demand)보다 비쌀 수 있습니다.
 - **유연성 부족**: 갑자기 트래픽이 몰려도 예약한 용량 이상은 처리할 수 없습니다.
 - **키워드**: 전용 용량 예약, 커스텀 모델 필수, 성능 보장(지연 시간 최소화)

모니터링 단계

- **Amazon Bedrock 모델 호출 로깅 (Model Invocation Logging)**
 - **개념**: Amazon Bedrock을 통해 오가는 모든 **요청(Input/프롬프트), 응답(Output/답변) 및 메타데이터**를 상세히 기록하는 기능입니다.
 - **장점**: * 모델이 실제로 어떤 답변을 내놓았는지 전수 조사할 수 있어 **감사(Audit)** 및 **문제 해결(Debug)**에 필수적입니다.
 - 모델의 성능(정확도)을 사후 평가하거나 유해 콘텐츠가 포함되었는지 확인하는 데 사용됩니다.
 - **저장 대상 (지원 리소스)**:
 - **Amazon S3**: 대량의 데이터를 장기간 보관하고 분석할 때 유리합니다.
 - **Amazon CloudWatch Logs**: 실시간 모니터링이나 특정 단어 감지 시 알람을 설정할 때 유리합니다.
 - **주요 특징 (시험 포인트)**:
 - **기본 설정**: 기본적으로는 **비활성화(Disabled)** 되어 있으므로 사용자가 직접 켜야 합니다.
 - **동일 리전**: 로그를 저장할 S3나 CloudWatch는 반드시 Bedrock과 **같은 리전, 같은 계정**에 있어야 합니다.
 - **CloudTrail과 차이**: CloudTrail은 '누가 호출했는지'만 기록하지만, 이 로깅은 '**실제 대화 내용(데이터)**'를 기록합니다.

- 키워드: 전체 요청/응답 데이터 수집, S3/CloudWatch 저장, 감사 및 문제 해결
- **On-Demand (온디맨드):** 기본 모델(FM)을 쓸 때 사용. 쓴 만큼 비용 지불.
- **핵심 기능:**
 - **Playgrounds:** 콘솔 안에서 코딩 없이 텍스트/이미지 모델 성능 테스트.
 - **Knowledge Bases:** 클릭 몇 번으로 **RAG** 환경 구축.
 - **Agents:** AI가 스스로 스케줄을 잡거나 API를 호출해 업무를 수행하게 함
 - AI 모델 - 백엔드와의 통합: 생성형 AI 모델과 백엔드 시스템 간의 간극을 메우는 중개자 역할 (에이전트는 말만 하는 게 아니라 직접 실행까지 해야 하니까...)
 - 자연어 프롬프트
 - API 요청으로 직접 변환: 외부 앱과 직접 대화
 - 데이터베이스 쿼리 생성: DB와 직접 대화
 - **특징:**
 - **멀티스텝 수행:** "보험금 청구해줘"라고 하면 [필수 서류 확인 → 누락 서류 요청 → 내부 시스템 등록]이라는 복잡한 단계를 알아서 계획하고 실행합니다.
 - **실시간 데이터 활용:** 미리 학습된 데이터가 아니라, 현재 시점의 날씨, 재고 상태, 예약 가능 여부 등을 외부 API로 가져와서 답변합니다.
 - **관리형 서비스:** 사용자들이 가져오신 지문처럼 서버 관리, 보안, 프롬프트 엔지니어링을 AWS가 다 알아서 해줍니다. 개발자는 AI에게 '무엇을 할지(Action Group)'만 알려주면 됩니다.
 - **Guardrails:** 안전 필터링, 모델의 입력(프롬프트)과 출력(응답) 사이에 설치하는 '실시간 보안 필터'입니다. 특정 주제나 문구를 차단하도록 제어할 수 있음
 - **PII(개인 식별 정보) 탐지:** 이름, 전화번호, 이메일 주소 등을 자동으로 인식해서 숨깁니다.
 - **정규 표현식(Regex) 지원:** 회사마다 다른 고유의 사번 양식이나 프로젝트 코드 같은 특수 기밀도 사용자가 직접 지정해서 가릴 수 있습니다.
 - **동적 마스킹(Masking):** 실시간으로 스캔해서 기밀 정보만 [CONFIDENTIAL] 처럼 가리거나 아예 삭제합니다.
- **PartyRock:** 콘솔 밖 별도의 웹사이트로 Bedrock 기반의 독립된 AI 앱 빌딩 서비스

*Amazon Q

- 정의: 사전 구성된 생성형 AI 애플리케이션을 만들 수 있도록 지원하는 생성형 AI 기반 어시스턴트, Bedrock 기반의 애플리케이션, 회사 내부의 데이터를 안전하게 학습하고 이해해서 답변을 해줌
- 특징
 - **사용자 정의 불가:** 어떤 모델(FM)을 쓸지 사용자가 고를 수 없습니다. AWS가 최적의 모델을 뒤에서 알아서 돌립니다.
 - **용도:** 코딩 지원(Q Developer), 비즈니스 데이터 검색(Q Business) 등 특정 목적에 바로 투입됩니다.
 - **강점:** 설치하고 데이터만 연결하면 바로 쓸 수 있는 '**Pre-packaged**' 앱입니다.
- 장점:

- **보안(Security):** 질문한 내용이 공개된 모델 학습에 사용되지 않습니다. (기업용이니깐요!)
- **권한 관리:** 사용자가 볼 권한이 없는 문서는 답변에 포함하지 않습니다. (IAM과 연동)
- **정확성:** 환각(Hallucination)을 줄이기 위해 기업 데이터에 기반한 답변을 우선시합니다.
- 종류: Developer - 개발자용, Business - 직원용, QuickSight - 분석가용
 - **Amazon Q Developer:** 개발자를 위한 비서 (코드 짜기, 인프라 구축) → IDE 환경 뿐만 아니라 AWS 관리 콘솔에서도 가능!
 - 코딩: 코드 생성, 버그 수정, 업그레이드
 - 클라우드 인프라 이해 및 관리: 자연어로 명령 가능 (배포 자체는 안됨)
 - 비용 분석 및 답변: 자연어로 질문 (비용 시각화는 불가)

→ Amazon Q Developer는 개발자의 코딩 작업에 맞는 코드 제안 및 권장 사항을 제공하여 개발자를 지원하도록 설계되었습니다. 방대한 데이터 세트로 학습된 머신 러닝 모델을 활용하여 코드 스니펫(복붙되는 코드 덩치)을 제안하고, 코드 효율성을 최적화하며, 개발자가 모범 사례를 따르도록 돕습니다.
 - **Amazon Q Business:** 일반 직원을 위한 비서 (회사 문서 검색, 데이터 분석), 완전 관리형 인프라 + 배포 및 구성이 쉬운 인터페이스. 단순히 답변만 하는 게 아니라 데이터 요약, 기술 문서 생성 등도 할 줄 안다!
 - 작동방식: RAG
 - 모드: 기업 데이터만 사용 or 모델 지식 + 기업 데이터 함께 사용
 - Plugin: Jira, Salesforce 등 상호작용 가능 c.f) Connector는 단순 읽기 기능
 - **Amazon Q Apps:** Q Business 안에서 사용자가 직접 만드는 '미니 앱'
 - **Amazon Q in Connect:** 고객 센터 상담원 옆에서 실시간으로 정답을 알려주는 AI 비서, 고객 상담자 전용!
 - Q in QuickSight: 데이터 시각화 생성, BI 대시보드 요약, 데이터 Q&A 등

Amazon Augmented AI (Amazon A2I)

- 정의: 아마존 증강 인공지능
 - **인간 검수 워크플로우 서비스:** 기계 학습(ML) 예측 결과에 **사람의 검토(Human Review)**를 쉽게 통합할 수 있도록 돕는 서비스입니다.
 - **Human-in-the-loop 구현:** AI가 100% 완벽하지 않다는 점을 인정하고, 중요한 결정 단계에 사람을 개입시키는 구조를 만들어줍니다.
- 특징
 - **신뢰도 기반 자동 트리거:** AI가 내놓은 답변의 **신뢰도 점수(Confidence Score)**가 미리 설정한 기준(예: 90점)보다 낮을 때만 사람에게 검토를 요청합니다.
 - **무작위 샘플링 검사:** AI가 아주 자신만만하게 대답한 내용이라도, 품질 관리를 위해 일정 비율을 무작위로 뽑아 사람에게 확인시킬 수 있습니다.

- **통합된 검수 인터페이스:** Amazon Rekognition(이미지), Amazon Textract(문서) 같은 서비스와 미리 연결되어 있어, 검수자가 화면에서 바로 틀린 부분을 수정할 수 있는 UI를 제공합니다.
- **유연한 검수 인력 선택:** 외부의 불특정 다수(Mechanical Turk), 우리 회사 직원(Private), 혹은 전문 업체(Vendor) 중 검수를 담당할 그룹을 자유롭게 지정할 수 있습니다.
- **사용 목적**
 - **정확도 향상:** AI가 실수할 확률이 높은 데이터를 사람이 직접 교정하여 시스템 전체의 정확도를 높입니다.
 - **규제 및 책임(Accountability):** 금융이나 의료처럼 법적 책임이 따르는 분야에서 AI의 판단을 사람이 최종 승인하도록 하여 안전장치를 마련합니다.
 - **지속적 학습 모델:** 사람이 수정한 데이터를 다시 학습 데이터로 사용하여, 시간이 지날수록 AI 모델 자체를 더 똑똑하게 만듭니다.

서비스	Amazon A2I	SageMaker Ground Truth
시점	모델을 사용(추론) 하는 단계	모델을 만들기(학습) 전 단계
목적	AI의 결과가 맞는지 검수/수정	데이터에 이름표(Label) 달기
역할	"AI가 좀 헛갈려 하네? 사람이 봐줄게"	"학습 데이터가 없네? 사람이 만들어줄게"

2) Amazon SageMaker (The Professional Way)

- 정의: 사용자가 맞춤형 모델을 구축, 학습 및 배포할 수 있도록 지원하는 완전 관리형 머신러닝 서비스
- 특징: 데이터 준비부터 모델 개발, 학습, 배포까지 머신러닝의 **전 과정(Lifecycle)**을 관리
- 요금제: 시간 단위 과금이며 On-Demand, Spot Instance, Saving Plans(1년 또는 3년)
 - c.f) BedRock은 토큰 기반 과금이며 서버리스라 스팟 인스턴스 없음

1. 데이터 진단, Exploratory Data Analysis

- 데이터 수집 직후 가장 먼저 수행하는 단계로 데이터가 어떻게 생겼는지 이해하는 단계, 시각화나 이상치 탐지 등
- **SageMaker Canvas (노코드/입문자용)**
 - **특징:** 코딩 한 줄 없이 마우스 클릭만으로 EDA와 데이터 준비를 끝낼 수 있는 서비스입니다.
 - **EDA 기능:** 데이터를 업로드하면 자동으로 분포를 그려주고, 어떤 변수가 정답에 큰 영향을 주는지 시각화해 줍니다.
 - **데이터 준비 기능:** "빈칸 채워줘", "중복 데이터 지워줘" 같은 명령을 클릭만으로 실행할 수 있습니다.
- **SageMaker Data Wrangler (전문가용/가장 매칭되는 서비스)**
 - **특징:** EDA(Exploratory Data Analysis)와 데이터 전처리에 특화된 **최고의 매칭 서비스**입니다.
 - **EDA 매칭:** 'Data Insights Report'를 클릭 한 번으로 생성하여 데이터의 품질, 이상치, 편향(Bias)을 한눈에 보여줍니다.

- **데이터 준비 매칭:** 300개 이상의 내장된 변환 로직(결측치 처리, 정규화 등)을 제공하여 데이터를 조리합니다.
- **노코드/로우 코드 :** 위 300개의 변환 로직이 그 핵심이며, 그 변환 로직을 사용자가 마우스로 클릭만 하면 됨

2. 데이터 준비

• SageMaker Data Wrangler → 데이터 전처리, 변환

- **정의:** 로우코드 노코드 툴(LCNC)로 데이터 전처리, 변환, 시각화에 특화된 도구 웹 UI, **시각적 데이터 분석** (초보자용) → 데이터 전처리에 특화)(비어 있는 값 채우기, 이상한 값 제거하기 등)
- **역할:** 데이터 전처리(Cleaning), 변환(Transforming), 시각화에 특화된 도구입니다.
 - **데이터 가져오기 및 시각화:** 공급망 데이터를 불러와서 데이터의 분포를 즉시 확인합니다.
 - **데이터 품질 검사:** 결측치(Missing values)나 이상치(Outliers)가 있는지 보고 바로 수정합니다.
 - **데이터 분할 (Split Data):** 클릭 몇 번으로 **훈련(Train)**, **검증(Validation)**, **테스트(Test)** 세트를 원하는 비율로 정확히 나눕니다.
 - **균형 잡기 (Balance Data):** 앞서 언급했듯이, 특정 공장의 데이터가 부족하면 오버샘플링(Oversampling) 등으로 균형을 맞춥니다.
- **한계:** 데이터를 예쁘게 다듬어 주긴 하지만, 모델을 직접 학습시키고 배포까지 완료하는 기능이 메인이 아닙니다. 요리로 치면 '재료 손질'까지만 해주는 도구예요.
- **대상:** 코딩을 최소화하고 싶은 **데이터 사이언티스트나 분석가**가 주로 씁니다.
- **하는 일:** Clarify가 찾아낸 편향을 해결하기 위해, 부족한 데이터를 복사해서 채우거나(Oversampling), 너무 많은 데이터를 지워서(Undersampling) **데이터셋의 균형(Balance)**을 맞춥니다.
- **키워드:** **Balancing(균형 맞추기)**, **Transform(변형)**, **Data Preparation(데이터 준비)**.
- **비유:** X-레이 결과를 보고 실제로 뼈를 맞추는 '**수술(치료)**'입니다.

→ c,f) 편향을 진단해주는 건 Clarify 이지만, Data Wrangler가 편향을 고쳐서 수정해줌

• SageMaker Ground Truth

- **정의:** 머신러닝 모델 학습에 필요한 ****고품질의 라벨링된 데이터셋(Labeled Dataset)****을 구축하도록 돕는 완전 관리형 서비스입니다.
- **역할:** 텍스트, 이미지 등 다양한 데이터에 대해 사람이 직접 주석을 다는 'Human-in-the-Loop' 기능을 제공하며, RLHF를 위한 데이터 제작에 필수입니다.

- **한계:** 작업 인력이 필요하므로 비용과 시간이 발생하며, 라벨링 품질이 사람의 숙련도에 의존합니다.
- 자동 데이터 라벨링: 사람이 모든 데이터를 라벨링하는 대신, 모델이 일부를 먼저 학습하여 **쉬운 데이터는 자동으로 라벨링**하고, **어려운 데이터만 사람(Worker)에게 전달**하여 비용과 시간을 70% 이상 절감합니다.
- **대상:** 대량의 미라벨링 데이터를 가진 ML 엔지니어 및 데이터 팀.
- **키워드:** 데이터 주석(Annotation), 라벨링, Human-in-the-Loop(학습 전), RLHF

- **SageMaker Clarify (훈련 전 단계)**

- **역할:** 학습 데이터 세트에 특정 성별, 인종 등 **편향(Bias)**이 포함되어 있는지 통계적으로 분석합니다.
- **한계:** 편향을 '발견'하고 보고서를 낼 뿐, 데이터를 직접 수정해주지는 않습니다.
- **대상:** 데이터의 공정성을 확보해야 하는 데이터 사이언티스트 및 컴플라이언스 담당자.
- **키워드:** 훈련 전 편향 감지(Pre-training Bias), 데이터 공정성, 공정성 지표

- **Studio Classic:** 대규모 데이터 처리, 노트북 환경, 대화형 세션, IDE > 최근에 JupyterLab 기반의 SageMaker Studio를 강조

- 빅데이터 처리를 위해 EMR이나 Glue와 연결해서 쓰기도 함

- **Processing API:** 스크립트 실행, ML Framework(PyTorch/MXNet/TensorFlow 활용) 완전 관리형 환경 (개발자용)

1. 특징 관리

- **SageMaker Feature Store: 학습-서빙 방지 목적,** 모델이 학습하거나 예측할 때 사용하는 가공된 입력 데이터 보관용 박스
 - Feature란: ML모델의 입력 데이터셋에서 정답을 제외한 모든 정보
 - 목적: 중복 작업 방지(여러 팀이 협업할 때, 겹치지 않도록 태깅 및 인덱싱), 학습-서빙 불일치 방지
 - 모드:
 - 온라인 스토어: 실제 서비스에서 추론할 때 사용, 아주 빠름
 - 오프라인 스토어: 모델을 학습시킬 때 사용, 대용량 데이터 보관
 - 한계: 인사이트까지는 스스로 주지 못함
 - 소스: S3, Redshift, Kinesis, Streaming Source
 - C.F) **특징 때문에** 결과가 나온다는 인사이트를 주는 건 Clarify임 (샤플리값)

1. 모델 빌드 및 학습

- **SageMaker Canvas** → AI를 만드는 도구, ML 모델 구축 및 예측 특화
 - **역할:** 코드 작성 없이(노코드) 마우스 클릭만으로 비즈니스 예측 모델을 만드는 No-code 서비스입니다. 최근에는 생성형 AI 모델(FM)을 테스트하고 비교하는 기능도 포함되었습니다.

- **한계:** 정해진 워크플로 안에서 작동하므로, 복잡한 커스텀 알고리즘이나 세밀한 모델 튜닝은 어렵습니다.
- **대상:** 데이터 과학 지식이 깊지 않은 비즈니스 분석가 및 도메인 전문가
- **키워드:** No-code ML, 비즈니스 분석가용, 클릭 기반 예측

○ SageMaker Autopilot

- **역할:** 데이터를 넣으면 알고리즘 선택, 하이퍼파라미터 튜닝 등 ML 전 과정을 자동으로 수행하여 최적의 모델을 생성(AutoML)합니다. Clarify를 활용해 예측 방식에 대한 통찰력도 제공합니다.
- **한계:** 주로 정형 데이터(표 형식)에 강점이 있으며, 고도로 전문화된 딥러닝 아키텍처 설계에는 제한적입니다.
- **대상:** 빠르게 고성능 모델을 구축하려는 개발자 및 ML 엔지니어.
- **키워드:** AutoML, 자동 모델 생성, 투명한 모델 분석

○ SageMaker JumpStart

- **정의:** 클릭 몇 번으로 최신 오픈 소스 모델(Llama, Mistral 등)이나 AWS 내장 알고리즘을 즉시 배포하고 미세 조정(Fine-tuning)할 수 있는 **모델 허브(모델 + 데이터셋 + 인프라 셋)**입니다.
- **특징:** **인프라 기반**입니다. 모델의 소스 코드(노트북)와 가중치를 내 계정의 **SageMaker 인스턴스**에 직접 배포해서 씁니다. 즉, 모델을 내 손안에 두고 뜯어고치고 싶을 때 사용합니다.
- **기능:**
 - **평가 및 비교 (Select):** 품질(Quality)과 책임(Responsibility) 지표를 보고 내 프로젝트에 딱 맞는 모델을 고를 수 있습니다.
 - **맞춤 설정 (Customizable):** 남이 만든 모델을 그대로 쓰는 게 아니라, **내 데이터(Your Data)**를 부어서 내 입맛에 맞게 튜닝(Fine-tuning)할 수 있습니다.
 - **공유 및 제어 (Governance):** 조직 내에서 모델과 노트북을 공유하고, 관리자는 보안이나 정책상 위험한 모델을 안 보이게 통제할 수 있습니다.
- **키워드:** 사전 학습된 모델(Pre-trained models), 원클릭 배포, 파운데이션 모델(FM) 허브.

○ Built-in Algorithms: AWS가 이미 짜놓은 최적화된 알고리즘 사용

○ Training Job: 인프라 자동 관리, S3에 결과 저장, 커스텀 알고리즘, 비용 효율적 (학습이 끝나면 컴퓨터를 자동으로 반납함)

- **키워드:** 스팟 인스턴스

2. 모델 최적화 및 평가

○ SageMakerExperiment: 데이터, 알고리즘, 매개변수의 다양한 조합을 시도할 때 각 시도가 모델의 정확도에 어떤 영향을 주었는지 추적하고 비교함

- **Automatic Model Tuning:** 모델의 성능을 결정하는 설정값인 '하이퍼 파라미터'를 자동으로 변경해가며, 수많은 학습 작업을 실행하고 가장 좋은 성적을 낸 모델을 찾는 기능
- **SageMaker Model Card: 개발, 검증, 운영 환경에서의 모델 설명서**
 - **역할:** 모델의 용도, 성능 지표, 학습 상세 정보를 문서화하여 관리합니다. 모델의 용도 및 위험 등급, 학습 세부 정보 및 지표, 평가 결과 및 관찰 내용, 고려 사항, 권장 사항 및 사용자 지정 정보와 같은 추가 정보를 카탈로그에 포함할 수 있습니다
 - **키워드:** 거버넌스, 문서화, 모델 스펙 관리

3. 배포 및 추론

- 배포 옵션: 실시간 추론(항상 켜져 있는 서버), 배치 추론(대량 데이터를 한꺼번에 처리하고 서버리를 끄), 서버리스 추론(평소엔 서버가 없다가 요청이 올 때만 잠깐 켜짐)
- **Inference Recommender**
 - **역할:** 모델 성능과 비용을 고려하여 가장 가성비 좋은 인스턴스 타입 (CPU/GPU 등)을 부하 테스트를 통해 추천합니다.
 - **한계:** 테스트 당시의 설정값을 기준으로 하므로 실제 운영 트래픽 변화에 따른 유동적인 대응과는 차이가 있을 수 있습니다.테스트를 위해 실제 인스턴스를 띄우므로 비용이 발생할 수 있음
 - **대상:** 배포 인프라 비용을 최적화하려는 운영팀(DevOps/MLOps).
 - **키워드:** 비용 효율성, 인스턴스 추천, 가성비 추론.
- **Amazon Augmented AI (A2I)**
 - **역할:** AI의 추론 결과가 불확실할 때(신뢰도 점수가 낮을 때), **자동으로 사람에게 검토를 요청**하는 워크플로를 제공합니다.
 - **한계:** 이미 나온 결과에 대한 검토이므로 실시간 응답 속도가 느려질 수 있습니다
 - **대상:** AI의 오진이 위험한 비즈니스(의료, 법률, 보안) 운영자
 - **키워드:** 저신뢰 예측 수동 검토, **Human-in-the-Loop(추론 단계)**
- **SageMaker Model Monitor**
 - **역할:** 프로덕션 환경에서 운영 중인 모델의 품질을 지속적으로 감시합니다. 데이터의 특성이 변하거나(Data Drift) 모델 성능이 저하되는 것을 감지합니다.
 - **한계:** 성능 저하를 감지하고 알릴 뿐, 자동으로 모델을 재학습시키
 - c.f) ModelCard: ML 모델에 대한 주요 세부 정보(사용 방법, 학습 및 성능에 대한 정보, 모델의 용도 및 위험 등급 등)를 문서화함

4. 모니터링

- **SageMaker Model Monitor**
 - **역할:** 프로덕션 환경에 배포된 모델의 품질을 지속적으로 감시합니다. 시간이 지나면서 데이터의 성격이 변해 모델이 멍청해지는 현상을 잡아냅니다.

- **한계:** 성능 저하를 감지하고 알람을 보낼 뿐, 모델을 자동으로 수정하거나 재학습시키지는 않습니다.
- **대상:** 모델이 배포된 후에도 예측 정확도를 일정하게 유지해야 하는 MLOps 엔지니어.
- **키워드:** 데이터 드리프트(Data Drift) 감지, 품질 모니터링, 실시간 감시, 개념 드리프트(Concept Drift)

○ SageMaker Model Dashboard

- **핵심:** 운영 현황과 인프라 자원 관리 (주로 운영할 때)
- **정의:** 내 계정에 있는 모든 모델의 현황을 한눈에 보고 검색할 수 있는 **중앙 집중식 포털**
- **기능:**
 - **모델 추적:** 어떤 모델이 실시간 엔드포인트에 떠 있는지, 어떤 게 배치 (Batch) 작업 중인지 확인
 - **성능 감시:** CPU, GPU, 메모리 같은 **인프라 자원** 사용량을 실시간으로 체크
 - **통합 관제:** SageMaker Model Monitor와 연결하면 실시간 데이터에 대한 모델의 **예측 성능(정확도 하락 등)**까지 한꺼번에 모니터링 가능

○ SageMaker Clarify (모니터링 단계) → AI를 검사하는 도구

- **핵심:** **편향성(Bias)**과 **설명 가능성(Explainability)** 분석
- **역할:** 훈련 데이터뿐만 아니라, **실제 운영 중에 들어오는 데이터**에서도 편향이 발생하는지 실시간으로 감시합니다. 또한, 운영 중인 모델의 특정 예측 결과에 대해 다시 한번 설명을 제공합니다.
- **한계:** 모니터링 설정 시 컴퓨팅 자원이 추가로 소모될 수 있습니다.
- **대상:** 배포 후에도 공정성과 투명성을 지속적으로 모니터링해야 하는 규제 준수 담당자.
- **키워드:** **운영 중 편향 모니터링(Bias Monitoring), 실시간 설명 가능성**
 - 모델 개발 및 추론 과정에서 입력 특징이 모델 예측에 어떻게 기여하는지 설명합니다.
- **핵심 기술:** SHAP and PDP > 설명 가능성 **1. SHAP (Shapley Values샤플리 값, 로컬 설명)**
 - **정의:** 게임이론을 만든 샤플리가 만든 개념으로, 단순히 기여도를 나눌 수 없으니 모든 조합의 경우의 수를 따져서 애가 새로 들어왔을 때 결과가 얼마나 변했는지를 보는 값인 한계 기여도를 평균낸 것
 - $\text{한계 기여도} = (\text{나를 포함한 팀의 성과}) - (\text{나를 뺀 팀의 성과})$
 - **핵심:** "**특정 데이터 하나**"에 대한 결과가 왜 그렇게 나왔는지 설명
 - **방식:** 게임 이론 기반으로 각 피처(Feature)가 최종 결과에 기여한 '공로'를 수치화

- **비유:** 축구 경기 승리 후 "미드필더인 A 선수는 30%, 공격수인 B 선수는 10% 기여했다"고 개별 분석하는 것
- **키워드:** 개별 예측 설명, 기여도 산출, Local Interpretability

2. PDP (Partial Dependence Plot, 글로벌 설명)

- **핵심:** "전체 데이터셋"에서 특정 피처가 모델 결과에 어떤 영향을 주는지 경향 파악
- **방식:** 다른 피처들은 고정하고, 타겟 피처의 값만 바꿔가며 결과의 평균적인 변화를 그래프로 시각화
- **비유:** "훈련 시간이 늘어날수록 전반적으로 득점력이 올라간다"는 통계적 경향을 그래프로 그리는 것
- **키워드:** 전체 모델 행동 이해, 마진 효과, 글로벌 트렌드, Global View

○ Amazon CloudWatch (SageMaker 통합)

- **역할:** SageMaker의 모든 지표(CPU/메모리 사용량, 호출 횟수, 오류율)를 로그로 수집하고 시각화합니다. 특정 임계치를 넘으면 알림을 보냅니다.
- **한계:** 모델 내부의 논리적 문제(정확도 하락 등)보다는 인프라적인 상태 감시에 더 가깝습니다.
- **방식:** CloudWatch Metrics(수치 측정) + Logs(저장) + Alarms(특정 조건 시 알림 발송)
- **대상:** 전체 시스템의 안정성과 리소스 사용량을 관리하는 IT 운영자
- **키워드:** 로그 수집, 메트릭 모니터링, 대시보드, 알람 설정

○ AWS CloudTrail (for Amazon Bedrock)

- **개념:** Amazon Bedrock을 포함한 모든 AWS 리소스에서 발생하는 **API 활동 및 사용자 호출 기록**을 캡처하고 기본적으로 90일 간 보관하는 서비스입니다. (일종의 'CCTV 기록') *90일 이상 로그를 보존하거나 감사 목적으로 저장하려면 S3 버킷 또는 CloudWatch Logs로 이벤트를 전송하는 트레일을 생성해야 합니다.
- **핵심 역할 (감사 및 보안):**
 - 누가(IAM 사용자), 언제, 어떤 API(예: InvokeModel , CreateProvisionedModelThroughput)를 호출했는지 기록합니다.
 - 규정 준수(Compliance) 확인 및 보안 사고 발생 시 추적 용도로 사용됩니다.
- **중요한 한계 (시험 단골 오답):**
 - **입출력 데이터 캡처 불가:** 프롬프트에 입력한 텍스트나 모델이 내뱉은 답변(실제 데이터)은 로그에 남기지 않습니다.
 - **성능 모니터링 부적합:** 모델이 얼마나 빠르게 답했는지(Latency)나 토큰 사용량 등 상세 성능 지표는 관리하지 않습니다.
- **키워드:** API 호출 기록, 감사(Audit), 보안 관리, 액세스 추적

- **SageMaker Model Registry**

- **역할:** 모델의 버전(v1, v2)을 관리하고, 운영 배포 전 승인(Approve/Reject) 프로세스를 관리하는 중앙 저장소입니다.
- **대상:** MLOps 팀
- **키워드:** 버전 관리, 모델 계보(Lineage), 승인 워크플로

- **Amazon SageMaker Studio**

- **역할:** 개발자가 머신러닝 모델을 만들기 위해 사용하는 통합 환경
- **환경:** 머신러닝에 필요한 모든 프로그램이 모여 있는 공간 ① **JupyterLab** (주피터랩) — "머신러닝의 표준 스케치북"
- **특징:** 코드를 한 줄 쓰고 바로 실행 결과를 확인할 수 있습니다. (그래프도 바로 보임)
- **용도:** 데이터 분석, 시각화, 실험적인 모델링.
- **시험 포인트:** 가장 많이 쓰이는 기본 도구입니다. "대화형(Interactive)" 코딩이라는 말이 나오면 주피터랩입니다.

- ② **Code-OSS (VS Code 기반) — "전문 개발자의 연장통"**

- **특징:** 우리에게 익숙한 **Visual Studio Code(VS Code)**의 오픈소스 버전입니다.
- **용도:** 단순한 실험을 넘어, 실제 서비스로 배포할 복잡한 소프트웨어를 개발할 때 씁니다.
- **시험 포인트:** "VS Code와 유사한 환경"이나 "엔터프라이즈급 개발"이라는 말이 나오면 이쪽입니다.

- ③ **RStudio — "통계학자의 전문 도구"**

- **특징:** 'Python' 대신 'R'이라는 프로그래밍 언어를 쓰는 사람들을 위한 곳입니다.
- **용도:** 고도의 통계 분석이나 학술적 연구.
- **시험 포인트:** 지문에 "R 언어를 사용한다"는 말이 나오면 무조건 RStudio가 답입니다.

보안/감사 서비스

- **Inspector:** 소프트웨어 취약점 진단, 자동화된 보안 평가 서비스
- **Config:** 히스토리 및 기록 '리소스', 리소스 구성 평가, 감사 및 검토하도록 함(구성 규정 준수, 모범 사례 시행)
- **Audit Manager:** 감사 증거 수집, 자동 감사 도구(지속적 감시, 자동 증거 수집, 위험 평가)

- **특징:**
 - **지속적 감시 (Continuous):** 설정해두면 24시간 내내 AWS 계정을 돌며 보안 상태를 체크합니다.
 - **자동 증거 수집 (Automated Evidence):** 규정에 어긋나는 설정이 있는지 스냅샷을 찍어 증거로 남깁니다. (사람이 일일이 캡처할 필요가 없음!)
 - c.f) Inspector: 이걸 자동 증거 수집 안되고 보안 평가 서비스임
 - **위험 평가 (Risk Assessment):** 금융권의 까다로운 규제(예: GDPR, HIPAA 등)를 기준으로 우리 시스템의 점수를 매겨줍니다.
- **Artifact:** 인증서, 규정 준수 보고서 포털(다운로드 하는 곳)
- **VPC (Virtual Private Cloud):** AWS 안에서 나만의 **가상 네트워크 망**을 만드는 것입니다. 다른 사람의 자원과 내 자원을 격리하는 울타리 역할을 합니다. 'Logically Isolated Network'
 - **특징:**
 - **End Point:** 인터넷 없이 AWS에 직접 연결하는 전용 통로
 - **Gateway Endpoint:** S3, DynamoDB 딱 2개뿐 >무료
 - **Interface Endpoint:** 유료, 대부분의 AWS 서비스(예, S3 등) 로 PrivateLink 기술을 사용하여 연결됨
- **IAM Identity Center: 중앙 집중식 액세스 관리:** 수많은 AWS 계정과 어플리케이션(Amazon Q, Slack, Office 365 등)에 대한 사용자 접속 권한을 **한 곳에서** 관리하는 서비스입니다.
 - **특징:**
 - **Single Sign-On (SSO):** 사용자가 한 번의 로그인으로 허용된 모든 서비스에 접근할 수 있게 해줍니다.
 - **외부 디렉터리 연결:** 회사가 이미 쓰고 있는 **Active Directory(AD)**나 **Okta** 같은 기존 사용자 명단을 그대로 연결해서 쓸 수 있습니다.
 - **Amazon Q Business 필수 조건:** Amazon Q Business 같은 최신 AI 서비스를 쓰려면, 누가 질문을 하는지 식별해야 하므로 반드시 IAM Identity Center 인스턴스가 설정되어 있어야 합니다. (일반 IAM 사용자는 바로 연결이 안 됩니다.)
 - **계정 및 조직 수준 지원:** 하나의 AWS 계정뿐만 아니라, 기업 전체 조직 (AWS Organizations) 단위로 통합 관리가 가능합니다.
 - c.f) IAM은 단일 AWS 계정 내부 전용, 이걸 여러 AWS 계정 + 외부 앱 통합 (더 최신)
- **Watermark Detection(BedRock 기능): Titan 모델이 만든 이미지인지 식별 (투명성 확보)**
 - **생성 단계:** 사용자가 Amazon Bedrock에서 **Titan Image Generator** 모델로 이미지를 만들면, 모델이 이미지 픽셀 속에 눈에 보이지 않는 아주 미세한 신호(워터마크)를 자동으로 심습니다.
 - **감지 단계:** 나중에 어떤 이미지를 보고 "이거 AI가 조작한 거 아냐?"라는 의심이 들 때, Bedrock의 **워터마크 감지 기능**을 쓰면 이 숨겨진 신호를 읽어서 "네, 이걸 **Titan 모델이 만든 이미지입니다**"라고 확인해 줍니다.

지도 학습 테크닉

테크닉이란 문제를 해결하는 **원리와 논리적 방식**입니다.

지도 학습 = 선형 회귀 + 로지스틱 회귀 + 의사결정 나무 + 신경망 +
거리기반 학습 <<

1. 선형 회귀 (Linear Regression): "연속적인 수치 맞추기"

- 방식: 데이터들의 분포를 가장 잘 설명하는 '직선' 하나를 긋는 방식입니다.
- 특징: 정답이 0.5, 10, 100처럼 **연속된 숫자**일 때 씁니다. (예: 공부 시간에 따른 시험 점수 예측)
- 결정 방식: $y = ax + b$ 라는 일차함수 식에서 최적의 기울기(a)와 절편(b)을 찾아냅니다.
- 알고리즘: Linear Learner

1. 로지스틱 회귀(Logistic Regression): "예/아니오 분류하기"

- 방식: 이름은 회귀지만 실제로는 '**분류**'에 씁니다. 직선이 아니라 **S자 곡선(Sigmoid, 로지스틱 함수)**을 그려서 확률을 계산합니다.
- 특징: 정답이 '**스팸이다/아니다**', '**합격/불합격**'처럼 딱 끊어지는 이진 분류일 때 씁니다.
 - C.F)Clustering은 정답이 아예 없는 상태에서 그냥 비슷한 놈들끼리 뭉치는 것
- 결정 방식: 결과값이 0.5보다 크면 1(참), 작으면 0(거짓)으로 판단합니다.
- 알고리즘: Linear Learner, SVM(Support Vector Machine)

1. 의사 결정 나무(Decision Tree): "스무고개 질문하기"

- 방식: "나이가 20세 이상인가?", "수입이 3천만 원 이상인가?"처럼 **질문을 던져 데이터를 쪼개나가는** 방식입니다.
- 특징: 사람이 이해하기 매우 쉽고(시각화 가능), 데이터에 수치와 카테고리가 섞여 있어도 잘 작동합니다.
- 결정 방식: 나무 가지가 뻗어나가듯 조건문(If-Then)을 통해 최종 결론에 도달합니다.
- 알고리즘: XGBoost, Random Forest

1. 신경망 (Neural Network): "인간의 뇌 흉내 내기"

- 방식: 여러 층의 뉴런(노드)을 쌓아서 아주 복잡한 관계를 학습하는 방식입니다. (딥러닝의 기초)
- 특징: 사진, 음성, 복잡한 텍스트처럼 **사람이 규칙을 만들기 어려운 비정형 데이터**에서 엄청난 성능을 냅니다.

- **결정 방식:** 입력 데이터에 수많은 가중치(매개변수)를 곱하고 더하는 과정을 반복해 정답을 도출합니다.
- 알고리즘: Factorization Machines, Deep AR

1. 거리 기반 학습 (Instance-based / Distance Methods)

- 방식: 데이터 간의 '거리'를 계산해서 끼리끼리 묶어주는 테크닉입니다.
- **구체적 알고리즘:**
 - **K-Nearest Neighbors (KNN):** "가장 가까운 이웃 k개가 누구니?"라고 묻는 유유상종 알고리즘. (Lazy Learner)
- **핵심 용도:** 소득/소비 패턴 분석, 간단한 이미지 분류

6. 행렬 분해 및 상호작용 (Matrix Factorization / Interaction)

데이터 사이의 숨겨진 상관관계(사용자와 아이템 사이의 관계 등)를 수치화하는 테크닉입니다.

- **구체적 알고리즘:**
 - **Factorization Machines (FM):** 빈칸이 많은(Sparse) 데이터에서도 "너 이거 좋아할 것 같은데?"라고 추천해 주는 엔진.
- **핵심 용도:** 넷플릭스 영화 추천, 전자상거래 상품 추천.

지도 학습 알고리즘

그 테크닉을 컴퓨터가 실행할 수 있도록 코드로 구체화한 것입니다. 같은 테크닉 안에서도 계산 방식이나 효율성에 따라 여러 알고리즘이 존재합니다.

- **Linear Learner:** 선형 학습자, 회귀와 분류를 둘 다 할 수 있음, 가장 기본적이고 연산 속도가 매우 빠름
 - 종류:
 - Support Vector Machine: 두 집단 사이의 '최적의 고속도로'를 뚫는 경계선 전문가
 - Logistic Regression: '스팸일 확률' 같은 분류에 특화
 - 예시: 내일 기온 맞히기, 이 메일이 스팸인지 맞히기
- **Factorization Machines:** 요인 분석기, 데이터 간 상호작용을 찾아내서 추천 시스템을 만드는 데 특화, 데이터에 빈칸이 많은 고차원 데이터 처리(Sparse Data)에 아주 강함
 - 예시: 넷플릭스 영화 추천
- **XGBoost:** 여러 개의 의사결정 나무를 합쳐서 결과를 냄, 정형 데이터에서 성능이 가장 좋아서 실무에서 압도적으로 많이 쓰임
 - 원조: Decision Trees
 - 예시: 고객 이탈 여부 예측, 신용 점수 계산
- **K-Nearest Neighbors(KNN):** 유유 상종, 새로운 데이터가 들어오면 그 데이터와 가장 가까이 있는 기존 데이터 k개를 찾아줌, Lazy Learner(미리 데이터를 학습해서 모델을 만들지 않고, 질문이 들어오면 처리)
 - 예시: 이미지 분류, 추천 시스템, 소득/소비패턴 비슷한 사람들의 신용 등급 참고

비지도학습 테크닉

비지도학습 = 클러스터링 + 차원 축소 + 이상 탐지 + 토픽 모델링 + 임베딩
<<

1. 클러스터링 (Clustering)

- **정의:** 정답(Label)이 없는 상태에서 데이터 간의 **유사성(거리)**만을 측정해 비슷한 특징을 가진 것끼리 그룹을 만드는 기술입니다.
- **비유:** 이름표 없는 수천 권의 책을 내용이 비슷한 것끼리 서가에 분류하여 꽂기.
- **핵심 용도:** 시장 세분화, 비슷한 뉴스 그룹화, 유전자 패턴 분석.
- **구체적 알고리즘:** **K-Means** (데이터를 K개의 중심점을 기준으로 묶는 가장 대중적인 방식)

2. 차원 축소 (Dimension Reduction)

- **정의:** 수많은 변수(Feature) 중 데이터의 본질적인 정보를 가장 잘 나타내는 **핵심 성분**만 남겨서 데이터의 복잡도를 줄이는 기술입니다.
- **비유:** 수천 페이지의 전공 서적을 핵심 키워드 중심의 10페이지 요약본으로 만들기.
- **핵심 용도:** 데이터 시각화(고차원 → 2D/3D), 머신러닝 전처리(계산 속도 향상), 노이즈 제거.
- **구체적 알고리즘:** **PCA (Principal Component Analysis)** (데이터의 변동성이 가장 큰 축을 찾아 압축하는 방식)

3. 이상 탐지 (Anomaly Detection)

- **정의:** 대다수의 데이터가 보여주는 일반적인 패턴에서 크게 벗어난 **희귀한 사례 (Outlier)**를 찾아내는 기술입니다.
- **비유:** 수만 개의 정상 제품이 지나가는 컨베이어 벨트에서 미세한 흠집이 있는 불량품 하나를 골라내기.
- **핵심 용도:** 카드 부정 결제 탐지(FDS), 서버 침입 탐지, 제조 공정 장비 고장 예측.
- **구체적 알고리즘:** **Random Cut Forest (RCF)** (데이터를 무작위로 쪼개어 고립시키기 쉬운 정도를 측정해 이상치를 판별하는 AWS 특허 방식)

1. 토픽 모델링 (Topic Modeling)

- **정의:** 방대한 문서 집합 속에서 공통적으로 나타나는 '**주제(Topic)**'들을 자동으로 찾아내어 문서를 분류하는 기술입니다.
- **비유:** 수만 장의 신문 기사를 읽지 않고도, "이 문치는 정치, 저 문치는 경제 기사네"라고 주제별로 키워드를 뽑아 분류하기.
- **핵심 용도:** 뉴스 카테고리 자동 분류, 고객 리뷰의 주요 불만 사항 파악, 대량의 문서 요약.
- **구체적 알고리즘:** **LDA (Latent Dirichlet Allocation)**, **NTM (Neural Topic Model)**

- 참고: AWS SageMaker는 이 두 가지를 모두 내장 알고리즘으로 제공합니다.

5. 임베딩 (Embedding)

- **정의:** 단어나 문장 같은 텍스트 데이터를 컴퓨터가 이해할 수 있는 **고차원의 숫자 벡터 (Vector)**로 변환하는 기술입니다. 단순히 숫자로 바꾸는 게 아니라, **의미가 비슷하면 숫자 사이의 거리도 가깝게** 만듭니다.
- **비유:** '왕'과 '왕비'라는 단어를 좌표 평면에 점으로 찍었을 때, '사과'보다는 두 점이 서로 훨씬 가까운 곳에 위치하도록 지도를 그리는 것.
- **핵심 용도:** 유사 문장 찾기, 텍스트 검색 엔진, 챗봇의 문맥 이해, 추천 시스템.
- **구체적 알고리즘:** **Word2Vec**, **Object2Vec** (AWS), **BERT** 등의 사전 학습 모델.

비지도학습 알고리즘

비지도 학습 = Clustering + Topic Modeling + Embedding + Anomaly Detection + Dimensionality Reduction

- Clustering
 - **K-Means:** 데이터를 K개의 그룹으로 나눔, 분산을 최소화해서 클러스터링하려는 것
 - **Latent Dirichlet Allocation(LDA):** 문서들 속에서 공통된 주제를 찾아냄
- Topic Modeling
 - **Latent Dirichlet Allocation(LDA):** 문서들 속에서 공통된 주제를 찾아냄
- Embedding: 데이터를 숫자로 변환
 - **Object2Vec:** 단어, 문서 혹은 사용자 ID 같은 객체를 수학적 벡터로 바꿈
- Anomaly Detection(이상 탐지)
 - **Random Cut Forest(RCF):** 데이터 흐름에서 혼자 튀는 이상한 값을 찾아냄
 - **IP Insights:** IP 주소와 사용자 행동 간 비정상적인 패턴을 분석함
- Dimensionality Reduction
 - **Principle Component Analysis(PCA):** 정보 손실을 최소화하면서 데이터의 변수 개수를 줄임

업로드중..

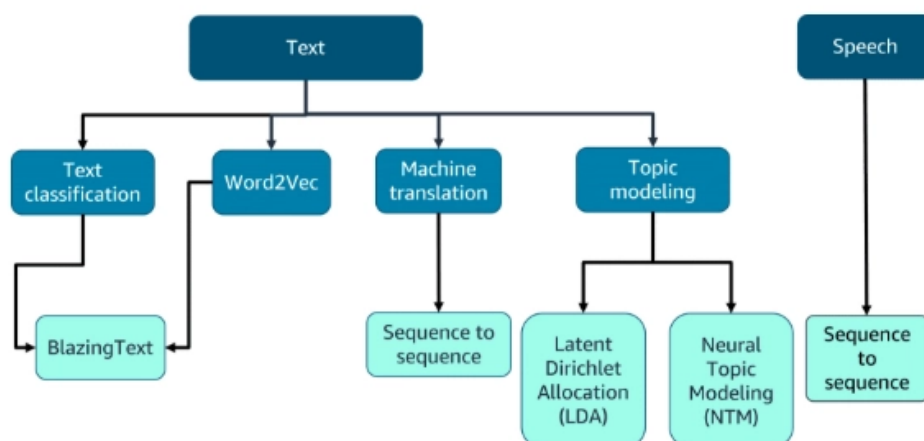
1. 이미지 처리

- 이미지 분류, 객체 탐지
 - **MXNet:** AWS가 공식적으로 지원하고 최적화한 머신러닝 프레임워크
 - **TensorFlow:** 구글에서 만든 가장 유명한 도구
- 의미론적 분할
 - **Fully Convolutional Network(FCN):** 이미지의 위치 정보를 끝까지 유지하며 픽셀을 분류하는 가장 기초적인 모델
 - **Pyramid Scene Parsing(PSP):** 주변 맥락을 파악해서 저건 물이고, 저건 하늘이다를 더 잘 구분하는 모델

- **DeepLab V3 + ResNet:** DeepLab V3은 이미지 픽셀 하나하나를 어떤 카테고리인지 분류하는 알고리즘의 머리 부분 + 이미지에서 특징을 추출하는 몸통
- 시계열 예측
 - **DeepAR:** 수천 개의 서로 관련 있는 시계열 데이터를 동시에 학습하여 미래를 예측함, 재귀 신경(RNN, Recurrent Neural Networks) 기반의 알고리즘

1. 텍스트 분석

- 텍스트 분류
 - **BlazingText:** 문장을 읽고, 그 문장이 어떤 카테고리에 속하는지 맞힘, 속도가 빛처럼 빠름
- Word2Vec: 단어를 숫자로
 - **BlazingText:** 단어를 의미가 담긴 벡터로 바꾸며, 비슷한 의미를 가진 단어끼리 가깝게 배치함
- 기계 번역
 - **Sequence to Sequence:** 한 문장을 입력 받아 완전히 다른 문장으로 바꿈, 번역이나 요약 시 사용
- 토픽 모델링
 - **Latent Dirichlet Allocation(LDA):** 단어의 통계적 분포를 보고 주제를 찾음, 가장 고전적이고 대중적
 - **Neural Topic Modeling(NTM):** 신경망을 사용하여 주제를 찾음, LDA보다 훨씬 복잡한 문맥을 파악할 때 유리함
- 음성 처리
 - **Sequence to Sequence:** 음성 데이터를 텍스트로 바꾸거나 반대로 바꿀 때 뼈대가 됨



- ML 실험관리(Experiment): 한 번에 끝나는 게 아니라 **데이터 + 알고리즘 + 파라미터**를 수만 번 조합해보는 **반복적(Iterative) 과정**입니다. 너무 많은 버전이 생기기 때문에 "어떤 설정일 때 가장 정확도가 높았지?"를 기억하기가 불가능에 가깝습니다.
 - **해결사: MLflow with Amazon SageMaker**

- **MLflow:** 머신러닝의 생애주기(실험, 재현, 배포)를 관리하기 위한 **오픈 소스** 플랫폼입니다.
- **SageMaker와의 시너지:** AWS의 강력한 인프라(SageMaker) 위에서 MLflow를 실행하여, 모든 실험 기록을 자동으로 저장하고 비교 분석합니다.
 - **주요 기능 (시험 및 실무 포인트)**
 - **Track (추적):** 학습 시 사용한 코드 버전, 데이터셋, 하이퍼파라미터 (설정값), 결과 지표(Accuracy 등)를 기록.
 - **Analyze & Compare (분석 및 비교):** 여러 번의 실행(Run) 결과를 차트로 시각화하여 어떤 모델이 최고인지 한눈에 파악.
 - **Register & Deploy (등록 및 배포):** 수많은 후보 중 '가장 잘난 모델'을 선정하여 모델 레지스트리에 등록하고 즉시 서비스로 배포.

기타 AWS 서비스

- **Rekognition:** 이미지/비디오 분석 (얼굴 인식, 사물 검출), 텍스트 추출(100자 이내)
 - 딥러닝 기술을 기반으로 이미지와 비디오 내의 객체, 사람, 텍스트, 장면 등을 자동으로 식별하고 분석하는 **완전 관리형 컴퓨터 비전(CV) 서비스**입니다. (CV 서비스라는 게 이미지 분류, 객체 탐지, 안면 인식 이런 거임)
 - **컴퓨터 비전(CV):** 이미지를 보고 그 안의 내용을 해석하여 의사결정을 내림 예) 자율 주행차, 얼굴 인식 잠금 해제 등
 - **주요 기능:**
 - **객체 및 장면 감지:** 사진 속 사물(자동차, 나무 등)이나 상황(해변, 결혼식 등)을 파악.
 - **얼굴 분석 및 비교:** 성별, 감정 상태(행복, 슬픔) 분석 및 두 사진 속 인물이 동일인인지 확인(사용자 인증).
 - **유해 콘텐츠 감지 (Moderation):** 부적절하거나 불쾌감을 줄 수 있는 이미지를 자동으로 필터링.
 - **텍스트 추출 (Rekognition Text):** 이미지 내의 문자를 읽어냄 (표지판, 문서 등).
 - **특징 및 장점:**
 - **ML 전문 지식 불필요:** 복잡한 알고리즘 설계 없이 API 하나로 고급 비전 기술 사용 가능.
 - **S3 연동:** Amazon S3에 저장된 대용량 데이터를 신속하게 분석.
 - **Custom Labels:** 기본 제공 기능 외에 우리 회사 로고나 특정 부품 등 **우리만의 데이터**를 추가 학습시켜 인식 가능.
 - **키워드:얼굴 인식, 유해 콘텐츠 감지, Custom Labels, API 기반 CV**
- **Polly:** 글자를 음성으로 (TTS).
 - SSML(Speech Synthesis Markup Language), 음성 합성 마크업 언어: AI 목소리에 연기력을 불어 넣음 → 속도, 높낮이, 음량, 강조, 호흡 → 음성 출력에 대한 사용자 지정 및 제어 목적(예. 속삭이듯이, 3초 쉬고, 강조해서 등),
 - c.f) 화자를 구분하는 게 아님 → 누가 말하느냐 X, '어떻게' 말하느냐!
 - 음성 선택 및 말하는 속도 및 높낮이까지 선택할 수 있음

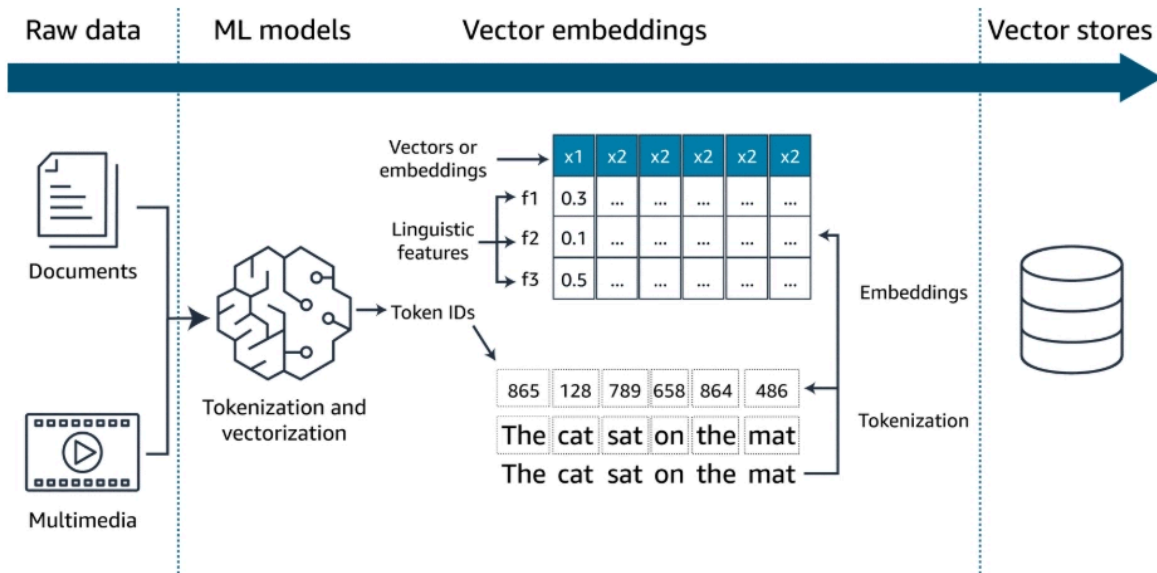
- **Transcribe:** 음성을 글자로 (STT), 사용량 기반 요금제 + 무료 사용자 지정 어휘 및 실시간 녹취, 비디오의 음성을 글자로 변환하는 것도 가능!
- **Comprehend:** 텍스트에서 감정(긍정/부정)이나 핵심 키워드 추출, 언어 감지, 문서/텍스트 분류, PII(개인 정보) 식별 및 마스킹 등을 하는 자연어 처리(NLP) 서비스. 의미와 맥락을 이해하며 뇌 역할을 함 + (텍스트) 개체 탐지/추도 함!
 - 활용: **개인식별정보(PII) 탐지 및 삭제**
 - 완전 관리형 서비스이므로 머신 러닝 전문 지식이나 맞춤형 모델 학습이 필요하지 않음!
 - 사용자 지정 분류: 고객 메일을 관련 부서에 자동 분류하여 전달할 수 있음
- **Kendra:** 기업용 지능형 검색 (RAG의 데이터 소스로 자주 쓰임)
 - 특징: 시맨틱 검색(semantic search) - 자연어 질문, 지능형 커넥터, 머신러닝 기반 재랭킹 - 의도 인식, 문맥적 이해, 자연어 처리
 - Kendra의 자연어 처리(NLP) 기능: 머신러닝을 사용하여 문맥과 의도를 해석함으로써 임상 기록과 같은 비정형 데이터의 관련성을 향상시킵니다.
 - Kendra에서 지원하는 데이터 소스: S3, Sharepoint, Google Drive
- **Lex:** 챗봇, 음성과 텍스트를 사용해 앱에 대화형 인터페이스를 구축하는 서비스, 자동 음성 인식 기능(ASR)이 있어서 말로 해도 다 알아 들음~ + 자연어 이해
 - 세션 속성: 상태 보존(이전 내용 기억), 개인화(개인화된 응답), 효율적인 대화 관리
→ 여러 단계에 걸쳐 맥락 유지, 사용자 정보 기억
 - CloudWatch Logs: Lex에서 대화 로그를 캡처하고 저장하는 강력한 솔루션으로 포괄적인 로깅을 제공함
 - 데이터 수집 및 저장 : **DynamoDB, 실시간으로 처리해야 하기 때문**
→ 다른 서비스도 추가한다면 Cloudwatch Logs가 과거 모든 기록을 저장하므로 도움이 됨
- **Personalize:** 실시간 개인화 및 추천 제공
- **DeepRacer:** 강화 학습(RL)을 배우기 위한 실물 교육용 도구(자율 주행 레이싱카)
 - 특징: wi-fi가 지원되는 실제 차량이며, 실제 트랙에서 자율 주행이 가능함(1/18 스케일 RC카) , 그리고 차가 없어도 시뮬레이터를 사용할 수 있음
- **Textract:** 스캔한 문서에서 텍스트, 필기, 레이아웃 요소 및 데이터를 자동으로 추출 (OCR)하는 머신 러닝(ML) 서비스입니다. 단순한 광학 문자 인식(OCR)을 넘어 문서에서 특정 데이터를 식별, 이해 및 추출합니다.
 - 한계: 맥락을 이해하면서 텍스트를 추출하는 건 아니고, 단순히 글자를 읽어오기만 하는 거
 - 사용 사례: 환자 데이터 추출, 대출 신청 처리(소득 및 재무 데이터 등 필수 정보 추출)
 - 신뢰도 점수: 내 답이 얼마나 정확한지 0~100% 사의의 숫자 지표 → 항상 추출한 텍스트와 함께 주는 정보
 - **양식 분석 (Form Analysis):** 모든 범용적인 양식에 해당되며, 문서의 구조를 파악해서 **키-값 쌍(Key-Value Pairs)**으로 묶어줍니다.

- Key-Value 쌍 추출: 송장/영수증과 같이 구조화된 문서에서 특정 정보를 식별하고 추출함
- **Mechanical Turk:** 컴퓨터보다 사람이 더 잘할 수 있는 작고 단순한 작업(HITs, Human Intelligence Tasks)을 전 세계의 분산된 작업자들에게 외주 줄 수 있는 플랫폼입니다.
- **Wavenet:** 음성 합성 및 오디오 신호 처리와 같은 특정 작업을 위한 오디오 데이터 생성에 최적화되어 있습니다
- **Kinesis:** 실시간 데이터 스트리밍 및 처리를 위한 서비스, 즉각적인 데이터 분석이나 실시간 모니터링이 필요할 때 적합함
- **Cognito:** 일반 외부 사용자(내부 직원 아님)의 인증, 권한, Identity 관리를 위한 서비스로 Lex의 짝꿍으로 잘 쓰임 '외부 사용자 관리'
 - c.f) IAM: 우리 회사 직원 대상의 사원증 같은 거 >
- **Fraud Detector:** 잠재적인 사기 행위 식별, 가짜 계정이나 결제 사기 등의 나쁜 짓들을 머신 러닝으로 잡아내는 서비스, 이진 분류 모델을 사용함(점수가 높을수록 사기일 확률이 높아짐)
 - 단계: 데이터 수집 → 모델 학습(aws 사기 탐지 전용 알고리즘) → 규칙 설정(90점 이상 등)
 - 데이터 저장: S3, 데이터가 거대하기 때문
 - 구성 요소: 거래 금액 + 기기 정보 + 사용자 위치
- **Athena:** SQL을 사용하여 S3의 데이터를 분석할 수 있는 대화형 쿼리 서비스

[제5장: 모델 최적화]

학습이 완료된 모델의 **성능(정확도)**은 최대한 유지하면서, **크기(용량)**를 줄이고 **속도(추론 시간)**를 높이는 단계입니다.

- 검색 증강 생성 : 벡터 임베딩, 벡터 데이터베이스
- 벡터 데이터 베이스 서비스:
 - Amazon OpenSearch Service (provisioned)
 - Amazon OpenSearch Serverless
 - pgvector extension in Amazon Relational Database Service (Amazon RDS) for PostgreSQL
 - pgvector extension in Amazon Aurora PostgreSQL-Compatible Edition
 - Amazon Kendra



- 에이전트, Agent: 파운데이션 모델(FM)의 추론 능력을 활용하여 복잡한 비즈니스 작업을 **엔드투엔드(End-to-End)**로 실행, **순환/반복**. 어떤 도구를 쓸지 스스로 결정함. 실행 및 조율의 역할

- 주요 용도 (Use Cases)

- **복잡한 작업 수행:** 단순한 질문 답변을 넘어 "휴가 신청을 시스템에 등록해줘" 같은 실행형 작업 수행, 트리거 기
- **외부 시스템 통합:** 사내 데이터베이스(DB)나 외부 API(날씨, 재고 확인 등)와 연결하여 실시간 정보 처리.
- **워크플로 자동화:** 여러 단계를 거쳐야 하는 비즈니스 프로세스를 모델이 스스로 판단해서 실행

- 작동 방식 (How it works):

- **추론(Reasoning):** 사용자의 요청을 분석하고, 이를 해결하기 위해 어떤 단계 (Step)가 필요한지 계획을 세웁니다.
- **액션 그룹(Action Groups):** 에이전트가 실행할 수 있는 '행동'들의 모음입니다. (예: Lambda 함수를 호출하여 DB 조회)
- **지식 기반(Knowledge Bases):** RAG(검색 증강 생성)를 통해 사내 문서 등을 참고하여 정확한 답변을 생성합니다.

- c.f) RAG: 딱 한 방향의 프로세스이며, 미리 연결된 데이터 소스에서 정보를 가져오는 보조 역할에 가까움

- **Fine-tuning (미세 조정):** 기본 모델은 매우 다재다능하지만, 광범위한 기능을 특정 응용 분야에 맞게 조정하거나 특정 영역에서 성능을 향상시키기 위해 미세 조정이 필요

- 방식:

1. 명령어 튜닝(Instruction tuning)

- **개념:** 프롬프트(질문)와 원하는 출력(답변) 쌍으로 구성된 데이터셋을 사용하여 모델이 **특정 명령어를 따르도록** 재학습하는 방식입니다.
- **장점:** 사용자 의도를 정확히 파악하며, 가상 비서나 챗봇 등 대화형 앱에서 답변의 질이 대폭 향상됩니다.

- **한계:** 고품질의 명령어 데이터셋 구축에 비용과 시간이 소요됩니다.
 - **키워드:** 지시 이행(Following instructions), 대화형 애플리케이션, 프롬프트-답변 쌍
2. 인간 피드백 기반 강화 학습(RLHF)
- **개념:** 데이터만 먹는 게 아니라, 결과에 대한 **피드백(보상/벌점)**을 통해 모델을 동적으로 다듬는 방식입니다.
 - **장점:** 사용자 만족도 같은 **직접적인 신호(Signal)**를 학습에 반영하므로, 빠르게 변하는 고객 취향에 최적화하기 좋습니다.
 - **한계:** 학습 과정이 점진적 학습보다 훨씬 복잡하고 설계가 어렵습니다.
 - **키워드:** 동적 최적화(Dynamic), 피드백 루프, 실시간 선호도 반영.
3. 특정 영역에 맞춘 모델 개발
- **개념:** 법률, 의료, 금융 등 **특정 산업 분야의 전문 데이터(코퍼스)**를 사용하여 해당 분야의 지식을 깊게 학습시키는 방식입니다.
 - **장점:** 특정 전문 분야에서 용어 이해도가 높아지며, 답변의 관련성과 정확도가 극대화됩니다.
 - **한계:** 해당 분야의 전문 데이터 확보가 어렵고, 범용적인 답변 능력은 다소 감소할 수 있습니다.
 - **키워드:** 산업 특화 지식, 전문 용어 이해, 맥락 고려.
4. 전이 학습(Transfer Learning)
- **개념:** 거대 데이터로 이미 학습된 모델(기초 모델)의 지식을 가져와서, 더 작고 구체적인 **작업(Task)**의 시작점으로 재사용하는 방법(재학습)입니다.
 - **정의:** 새로운 레이어를 추가하고 특정 데이터셋으로 재학습시켜 새로운 작업에 맞게 조정함으로써 사전 학습된 머신러닝 모델을 개선할 수 있는 기술
 - **장점:** 바닥부터 새로 학습시킬 필요가 없어 학습 시간을 최소화하고 매우 효율적으로 성능을 낼 수 있습니다.
 - **한계:** 원본 모델(Source)과 대상 작업(Target) 간의 연관성이 너무 낮으면 성능이 떨어질 수 있습니다.
 - **키워드:** 지식 재사용, 효율성, 시작점(Starting point)
5. 지속적 사전 학습(Incremental Training, Continued Pre-training)
- **개념:** 처음부터 다시 배우는 게 아니라, 기존 지식을 유지하면서 **새로운 데이터만 추가로 학습**하는 방식입니다. (업데이트 개념)
 - **장점:** 전체 데이터를 매번 처음부터 학습시킬 필요가 없어 시간과 비용이 절약됩니다.
 - **한계:** 단순히 데이터만 추가하는 것이라, 고객의 **실시간 선호도 변화나 대화 스타일의 미묘한 차이**를 즉각적으로 따라잡기엔 속도가 느립니다.
 - **키워드:** 지식 유지(Retaining), 부분 업데이트, 비용 효율성

모델 배포

- Self hosted API: 사용자가 인프라를 직접 구축하고 관리함. 주로 Docker 컨테이너를 사용함

- Managed API: aws가 책임져주는 방식으로 대표적으로 SageMaker

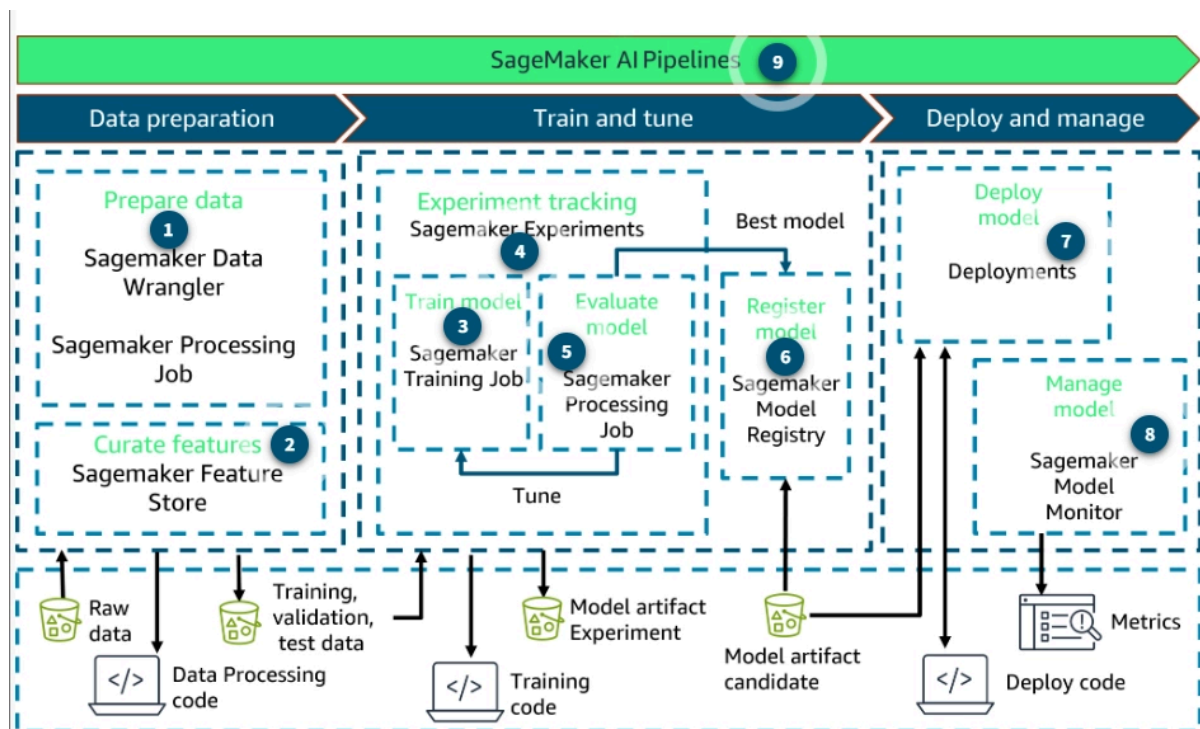
모델 추론

사용자의 질문(Prompt)에 답을 생성하는 실행 단계입니다.

- 동기(Sync): 즉시 대답 (실시간 챗봇)
- 비동기(Async): 번호표 뽑고 대기 (1GB 미만 큰 데이터)
- 배치(Batch): 한꺼번에 대량 처리 (수 GB 이상 거대 데이터)

MLOps

- 정의: 모델을 만드는 과정과 서비스로 운영하는 과정을 하나로 합친 것
- 3대 기둥
 - CI(Continuous Integration): 데이터나 모델 학습 코드가 바뀌면, 자동으로 테스트하고 문제가 없는지 확인하는 과정
 - CD(continuous Deployment): 검증된 모델을 자동으로 엔드포인트(서버)에 갈아 끼우는 과정
 - CT(Continuous Training): 지속적 학습으로, 새로운 데이터가 들어오면 모델이 다시 스스로 학습하는 과정
- 서비스
 - **SageMaker Pipelines**: 모델 제작 과정을 '워크플로'로 만들어 자동화합니다. (가장 핵심!)
 - **SageMaker Model Registry**: 버전별로 모델을 관리하고 승인합니다. (마치 모델 전용 도서관)
 - **SageMaker Model Monitor**: 배포된 모델의 성능이 떨어지지 않는지 24시간 감시합니다.



[제5장: Responsible AI]

최근 AWS가 가장 강조하는 **윤리와 안전** 파트입니다.

- 정의: 책임 있는 AI란 AI 시스템이 투명하고 신뢰할 수 있도록 보장하는 동시에 잠재적 위험과 부정적인 결과를 최소화하는 관행과 원칙
- 문서: AI Service Cards, AWS 서비스 자체의 투명성을 제공함
- 요소:
 1. **공정성**
 2. **설명 가능성**: 인공지능 모델이 내부 메커니즘과 결정에 대해 인간이 이해할 수 있도록 명확하게 설명하거나 정당성을 제시할 수 있는 능력
 3. **개인정보 보호 및 보안** : 개인이 자신의 데이터가 언제, 어떻게 사용될 수 있는지에 대한 통제권을 갖는 것, 승인되지 않은 시스템이나 사용자가 개인 데이터에 접근하지 못하도록 보장하는 것
 4. **투명도**: 이해관계자들이 시스템 사용에 대해 정보에 입각한 결정을 내릴 수 있도록 AI 시스템에 대한 정보를 제공하는 것
 5. **진실과 견고성**: 예상치 못한 상황, 불확실성 및 오류가 발생하더라도 인공지능 시스템이 안정적으로 작동하도록 보장하는 메커니즘
 6. **거버넌스**: 조직 내에서 책임감 있는 AI 관행을 정의, 구현 및 시행하는데 사용되는 일련의 프로세스
 - 투명성 메커니즘 구현
 - 명확한 책임 체계 구축(책임 소재를 명확히 규정), Human In the Loop를 강조함
 7. **안전**
 8. **제어 가능**
- 오용 및 리스크
 - Poisoning: 유해성, 모델의 훈련 데이터 세트에 악의적이거나 편향된 데이터를 의도적으로 삽입함
 - Hijacking, 하이재킹: 경로 탈취(목적 변경)이 목적, 정상적인 답변 경로를 중간에 낚아채서 엉뚱하거나 악의적인 목적으로 쓰이게 함. 프롬프트 자체에 특정 명령어를 삽입하여 생성 모델의 출력에 영향을 미침 (은근슬쩍 단소리 하기)
 - c.f) 탈옥: 안전 가드레일 무력화(금기 위반)이 목적임,
 - Exposure
 - Prompt Leaking: 프롬프트 유출, 모델에서 사용되는 다른 데이터를 노출시켜 모델의 작동 방식을 드러낼 수 있음
 - Jailbreaking, 탈옥: 안전 가드레일이나 금지 규칙을 무너뜨리는 데 집중하는 공격. 생성 모델이나 AI 비서에 구현된 제약 조건 및 보안 조치를 수정하거나 우회하여, 무단으로 접근하거나 기능을 사용하는 행위 (완전 나쁜 맘을 먹고 접근하기)

- AWS 서비스 도구:

1. 데이터 준비 및 가공 단계

- **SageMaker Data Wrangler:** "데이터를 요리하기 좋게 손질하는 도구"입니다. 특히 말씀하신 **불균형 데이터(데이터가 한쪽으로 쏠린 상황)**를 해결할 때 필수입니다.
- **SageMaker Ground Truth:** "사람의 손길(레이블링)"이 필요할 때 씁니다. 사진을 보고 "이건 고양이네"라고 직접 답을 달아주는 **Human-in-the-Loop**의 핵심입니다.

2. 모델 생성 및 분석 단계

- **SageMaker Autopilot:** 자동으로 모델을 만들어주는 도구
 - **자동 모델 구축:** 제공된 데이터 세트를 기반으로 적절한 머신러닝 알고리즘을 자동으로 선택합니다. 그런 다음 비교를 위해 여러 모델을 구축하고 학습시킵니다.
 - **하이퍼파라미터 최적화:** 각 모델의 하이퍼파라미터를 미세 조정하여 성능을 향상시키고 평가 지표를 기반으로 가장 성능이 좋은 모델을 선택합니다.
 - c.f) 모델이 학습 중에 스스로 하이퍼파라미터를 자동으로 바꾸는 건 아님
 - **배포 가능한 모델:** 실제 운영 환경에 바로 배포할수 있으며 투명성을 제공함
- **SageMaker Clarify:** 모델이 편향되지는 않았는지, 결과에 대한 설명이 가능한지 (Explainability)를 분석합니다.
 - 편향 감지
 - 설명 가능

3. 관리 및 문서화 단계 (Governance)

- **SageMaker Model Card:** 개발, 검증, 운영 단계에서의 환경"모델의 이력서(또는 설명서)"입니다. 모델이 어떻게 만들어졌고, 위험도는 어떤지 한눈에 보게 해줍니다.
 - **상세 기술 정보:** 학습 데이터 세트, 학습 메트릭(정확도 등), 평가 결과, 하이퍼파라미터 등의 **기술적 수치**를 포함합니다.
 - **위험 및 의도:** 모델의 **의도된 용도(Intended Use)**와 **위험 등급(Risk Rating)**을 명시하여 오남용을 방지합니다.
 - **함정:** 학습하지 않은 모델에 대한 정보도 포함 → why? 학습하지 않은 모델이지만 그 모델로 테스트를 해봤을 수도 있음
- **SageMaker Model Monitor:** 배포된 모델이 시간이 지나면서 바보가 되지는 않는지 실시간으로 체크
- **AI Services Card:** Amazon Rekognition, Amazon Textract 같은 **AWS의 기성 AI 서비스**들이 어떻게 설계되었고, 어떤 한계가 있는지를 AWS가 고객에게 공개하는

문서입니다.

- 거버넌스 측면: Model Card + Model Monitor + Role Manager

- c.f) Clarify는 분석 도구이지 거버넌스를 위한 도구는 아님

- Responsible Dataset

1. 포괄적이고 다양한 데이터 수집

2. 데이터 큐레이션: 모델이 정확하게 작동할 수 있도록 데이터에 레이블을 지정, 구성 및 전처리하는 과정

- 주요 단계:

1. 데이터 전처리: 데이터 클리닝, 정규화, 특징 선택과 같은 기법으로 데이터셋의 편향 제

2. 데이터 증강: 대표성이 부족한 집단의 새로운 인스턴스를 생성

3. 정기 감사: 편향 여부 확인

- 가치

- 투명성: 모델이 '어떻게' 의사결정을 내리는지에 대한 답을 제시, 감사에 도움

- 설명 가능성: 모델이 '왜' 그렇게 결정을 내렸는지에 대한 답을 제시함, 디버깅에 도움, 어떤 요소가 결과에 큰 영향을 주었는지 사후에 설명해주는 것. 모델이 그런 결정을 내린 과정과 이유에 초점

- 해석 가능성(Interpretability): 사람이 가중치와 특징을 기반으로 모델 출력을 해석할 수 있도록 함, 내부 매커니즘을 이해하는 상태

- 제어 가능성: 입력 데이터를 변경하여 모델을 얼마나 제어할 수 있는지로 측정됨

- 인간 중심의 디자인(Human Centered Design)

- Design for amplified: 중대한 상황에서 의사 결정자 지원, 스트레스 상황이나 압박감이 심한 환경에서 의사결정을 내릴 때 발생하는 위험과 오류를 줄이는데 중심

- Design for unbiased: 의사결정 과정, 시스템 및 도구의 설계가 결과에 영향을 미칠 수 있는 편견으로부터 자유롭도록 보장하는 것을 목표

- Design for Human and AI Learning: 인간과 AI 모두에게 유익하고 효과적인 학습 환경과 도구를 만드는 것을 목표

1. 데이터 레지던시 (Data Residency): "어디에 두는가?"

이것은 데이터의 '**물리적 주소**'에 대한 이야기입니다. API가 호출된 AWS 리전 내 데이터가 유지됩니다.

- **핵심:** 국가나 지역의 법률(GDPR, 국내 개인정보보호법 등) 때문에 데이터가 특정 국경을 넘지 못하게 관리하는 것입니다.

- **AWS 예시:** "우리 회사는 한국 고객 정보를 다루니까 반드시 **서울 리전(ap-northeast-2)** 내에만 데이터를 저장해야 해!"라고 설정하는 것이 데이터 레지던시를 준수하는 것입니다.

2. 데이터 리텐션 (Data Retention): "언제까지 두는가?"

이것은 데이터의 '유통기한'에 대한 이야기입니다.

- **핵심:** 법적 증빙을 위해 최소 5년은 보관해야 한다거나, 혹은 보안을 위해 30일이 지나면 영구 삭제해야 한다는 등의 정책입니다.
- **AWS 예시: * S3 Lifecycle Policy:** "6개월 지난 파일은 저렴한 Glacier로 옮기고, 5년 지나면 삭제해!"
 - **CloudWatch Logs:** "로그 데이터는 너무 많으니 30일만 보관하고 지워줘."

1. 데이터 로깅: 데이터를 체계적으로 기록. 입력 데이터, 모델 예측, 성능 지표와 같은 정보를 수집하여 AI 시스템의 동작을 추적하고 분석하는 것이 포함됩니다.

- 심층 방어
 - 단계:
 1. 입력 검증: 프롬프트 인젝션(악의적인 프롬프트)을 입구에서 컷트시킴
 2. 액세스 제어: 누가 모델을 사용하고 데이터를 수정할 수 있는지 엄격하게 관리함
 3. 지속적 모니터링: 실시간으로 시스템을 감시하다가 이상 징후가 보이면 즉시 알림

- **생성형 AI 보안 스코핑 매트릭스(Generative AI Security Scoping Matrix)**

- 정의: 생성형 AI 도입할 때 누가, 어디까지 보안을 책임져야 하는가를 정의한 지도
- 관리 영역:
 1. 거버넌스 및 컴플라이언스: AI 사용에 대한 사내 규칙과 보고 절차를 만드는 것
 2. 법적 및 개인 정보보호: 저작권 문제나 개인정보 보호법 등을 어기지 않는지 체크
 3. 위험 관리: 발생 가능한 위협을 미리 찾아내고 어떻게 막을지 고민하는 과정
 4. 통제: 위협을 막기 위해 실제로 설치하는 기술적 장치
 5. 회복탄력성: 시스템이 중단되지 않게 설계하고, 문제가 생겨도 금방 복구하도록 하는 능력
- 시나리오, Scoping:
 - SaaS 직접 사용: 책임이 가장 낮음
 - API 기반 앱 빌드
 - 파인 튜닝
 - 모델 자체 제작 : 책임이 가장 높음

- **Config:** AWS 리소스 구성을 평가, 감사 및 검토할 수 있도록 지원하는 서비스입니다. AWS 리소스 구성을 지속적으로 모니터링하고 기록하며, 원하는 구성에 대한 규정 준수 여부를 자동으로 확인할 수 있습니다.

1. 위협 탐지 (Threat Detection): "도둑이야!"

지금 당장 벌어지고 있는 '공격 행위'를 감시하는 것입니다.

- **성격:** 실시간 감시 (CCTV)
- **대상:** 외부 침입자, 악성 코드 실행, 이상 로그인 등
- **주요 도구:** **Amazon GuardDuty** (지능형 위협 탐지 서비스)
- **비유:** 우리 집 담벼락을 넘으려거나 문고리를 돌려보는 '**수상한 사람**'을 CCTV로 발견하고 즉시 알람을 울리는 것과 같습니다.

2. 취약점 관리 (Vulnerability Management): "자물쇠 고치자!"

공격이 들어오기 전에 우리 시스템에 있는 '**약점**'을 미리 찾아 보수하는 것입니다.

- **성격:** 사전 예방 (건물 안전진단)
- **대상:** 업데이트 안 된 소프트웨어, 잘못된 보안 설정, 열려 있는 포트 등
- **주요 도구:** **Amazon Inspector** (자동 취약점 스캐닝 서비스)
- **비유:** 도둑이 오기 전에 우리 집 창문 자물쇠가 헐겁지는 않은지, 뒷문이 열려 있지는 않은지 '**미리 점검하고 수리**'하는 것과 같습니다.

AI Practioner 오답 노트



HyunJin Na

나를 소개

0개의 댓글

댓글을 작성하세요

댓글 작성

관심 있을 만한 포스트

